

PostgreSQL I — A Topic-by-Topic Guide

Contents

1	Introduction and architecture	5
1.1	Scope and conventions	5
1.2	The architecture: processes and memory	5
1.3	The life of a query	6
1.4	Roadmap	6
2	Joins	7
2.1	Two orthogonal axes (flag this first)	7
2.2	The running dataset	7
2.3	CROSS JOIN — every pairing	8
2.4	INNER JOIN — matches only	8
2.5	LEFT, RIGHT, FULL OUTER — keep the unmatched	9
2.6	SEMI and ANTI joins — filter, don't combine	10
2.7	SELF join — a table joined to itself	10
2.8	LATERAL join — the right side sees the left	11
2.9	Non-equi (theta) and range joins	11
2.10	Physical execution: nested loop, hash, merge	11
2.11	Summary: all the join types	12
3	Connection pooling	13
3.1	Why pooling: one backend per connection	13
3.2	Where pooling lives	13
3.3	The three pooling modes	14
3.4	What transaction pooling breaks (and how to cope)	15
3.5	Sizing the pool	15
3.6	A concrete PgBouncer setup	15
3.7	Choosing a mode	16
4	The <code>psql</code> client	16
4.1	Connecting	16
4.2	Two kinds of input: SQL vs. meta-commands	16
4.3	Reading the prompt	17
4.4	Inspecting the database: the <code>\d</code> family	17
4.5	Output formatting: aligned, expanded, <code>\pset</code>	18
4.6	The query buffer: <code>\e</code> , <code>\i</code> , <code>\g</code>	18
4.7	Variables and interpolation	19
4.8	Importing and exporting: <code>\copy</code> vs. <code>COPY</code>	19
4.9	Measuring and watching	19

4.10	Scripting psql non-interactively	19
4.11	Customizing with .psqlrc	20
4.12	Meta-command quick reference	20
5	INSERT and COPY	20
5.1	INSERT: the basic forms	20
5.2	RETURNING: get data back from the write	21
5.3	Upsert: INSERT ...ON CONFLICT	21
5.4	COPY: bulk load and unload	22
5.5	COPY (server) vs. \copy (client)	22
5.6	Why COPY beats even a multi-row INSERT	23
5.7	Choosing, and loading fast	23
6	Ordering query results (ORDER BY)	24
6.1	Rows have no order until you ask (flag)	24
6.2	The basics: keys, direction, multiple columns	24
6.3	NULL ordering	24
6.4	Ordering by expressions, aliases, positions, collation	25
6.5	Top- <i>N</i> and pagination: ORDER BY with LIMIT	25
6.6	DISTINCT ON: first row per group	25
6.7	How ordering executes	26
6.8	Summary	27
7	The logical order of a SELECT	27
7.1	Written order is not evaluation order	27
7.2	The eight phases	27
7.3	Why the order matters: the consequences	28
7.4	Logical order vs physical execution	30
7.5	PostgreSQL-specific niceties	30
7.6	Reference: what each clause may use	30
8	Aggregate functions	30
8.1	The common aggregates	31
8.2	NULLs and the count() trio (flag)	31
8.3	GROUP BY: one value per group	32
8.4	FILTER: conditional aggregates	32
8.5	Aggregates with an internal ORDER BY	32
8.6	Ordered-set aggregates: median, percentiles, mode	32
8.7	Subtotals: GROUPING SETS, ROLLUP, CUBE	33
8.8	Aggregate vs. window function (same function, two behaviours)	33
8.9	A note on performance	33
9	The write-ahead log (WAL)	34
9.1	The problem: durability without flushing everything	34
9.2	The write-ahead rule (the invariant)	34
9.3	LSN: the WAL clock	35
9.4	The commit path: what fsync buys	35
9.5	Checkpoints	35

9.6	Crash recovery (REDO)	35
9.7	Full-page writes (torn-page protection)	35
9.8	Durability and detail knobs	36
9.9	What WAL also enables	36
9.10	Inspecting WAL	36
10	Triggers	37
10.1	The taxonomy: timing $\times$ event $\times$ level	37
10.2	Firing order	37
10.3	Inside the function: NEW, OLD, and TG_*	37
10.4	Return-value semantics	38
10.5	Worked examples	38
10.6	WHEN: fire only when needed	39
10.7	Statement-level triggers and transition tables	40
10.8	Constraint triggers (deferrable)	40
10.9	Caveats (flags)	40
10.10	Managing triggers	40
11	DELETE vs TRUNCATE	40
11.1	What each does	41
11.2	The key difference: storage	41
11.3	Side by side	42
11.4	DELETE in practice	42
11.5	TRUNCATE specifics	42
11.6	Both are transactional in PostgreSQL (flag)	43
11.7	When to use which	43
12	Foreign keys	43
12.1	Declaring a foreign key	43
12.2	The integrity rule and NULLs	43
12.3	Referential actions: ON DELETE / ON UPDATE	44
12.4	NO ACTION vs RESTRICT (the subtle one)	44
12.5	How a foreign key is enforced	45
12.6	Index your foreign-key columns (gotcha)	45
12.7	Deferrable constraints	45
12.8	Adding a foreign key to a big table	45
12.9	Managing	45
13	Transactions	46
13.1	BEGIN, COMMIT, ROLLBACK, autocommit	46
13.2	ACID, and how PostgreSQL provides each	46
13.3	Savepoints: partial rollback	46
13.4	MVCC: snapshots and row versions	46
13.5	Isolation levels	46
13.6	READ COMMITTED vs REPEATABLE READ	47
13.7	SERIALIZABLE and serialization failures	47
13.8	Row locking and lost updates	48
13.9	Long transactions and idle-in-transaction (gotcha)	48

13.10DDL is transactional (flag)	48
14 MVCC in depth	49
14.1 The tuple header	49
14.2 Visibility	49
14.3 Version chains and HOT updates	49
14.4 Dead tuples, bloat, and VACUUM	49
14.5 The visibility map	50
14.6 Transaction-id wraparound and freezing	50
14.7 Seeing it	51
15 Window functions	51
15.1 Anatomy: OVER (PARTITION BY ... ORDER BY ... frame)	51
15.2 Ranking functions	51
15.3 Offset and positional functions	52
15.4 Aggregates as windows: running totals, shares	52
15.5 Frames: ROWS vs RANGE (a gotcha)	52
15.6 Top- <i>N</i> per group	52
16 Table partitioning	53
16.1 Why partition	53
16.2 Declarative partitioning: RANGE, LIST, HASH	53
16.3 Partition pruning	54
16.4 Attaching and detaching	54
16.5 Indexes and constraints	54
16.6 Gotchas (flags)	55
17 The type system	55
17.1 Numeric: exact vs floating (flag)	56
17.2 Character: prefer text	56
17.3 Temporal: always timestampz	56
17.4 JSON and JSONB	56
17.5 Arrays	56
17.6 Ranges	57
17.7 Casting	57
17.8 Under the hood	57
18 User-defined types	57
18.1 Domains: a constrained existing type	57
18.2 Enums: a fixed set of labels	58
18.3 Composite types: a row type	58
18.4 Range types	58
18.5 Base types (the C route)	58
18.6 Making them first-class	58

19 Strings and text	58
19.1 Length and encoding	59
19.2 Building and measuring	59
19.3 Case and trimming	59
19.4 Slicing and finding (1-based)	59
19.5 Replace and translate	59
19.6 Pattern matching	60
19.7 Splitting and aggregating	60
19.8 Collation and case-insensitivity	60
19.9 Indexing pattern searches (performance)	60
20 Glossary of symbols and terms	60

1 Introduction and architecture

This is a topic-by-topic guide to PostgreSQL. Each chapter is a self-contained treatment of one subject — architecture, storage, transactions, indexing, joins, partitioning, connection pooling, replication, performance, and more — written so it can be read on its own and so new topics can be slotted in as the guide grows. This opening chapter sets the shared mental model everything else hangs on: the processes, the memory, and the path a query takes from text to rows.

1.1 Scope and conventions

- **Version.** PostgreSQL 16+; where behaviour is version-specific we say so.
- **Breadth over recipes.** The aim is understanding of how PostgreSQL works, chapter by chapter — not a cookbook. Performance and tuning are *one* chapter among many, not the whole story.
- **Prerequisites.** Comfort writing SQL (`SELECT`, `JOIN`, `GROUP BY`). We explain concepts and internals, not basic syntax.
- **Notation.** A relation (table/view/subquery result) is R or S ; its row count (cardinality) is $|R|$; a predicate’s selectivity is $s \in [0, 1]$. Other symbols are defined where introduced and collected in the glossary.

1.2 The architecture: processes and memory

PostgreSQL is a classic client/server system built on *processes*, not threads. Figure 1 is the map. A single *postmaster* listens for connections and forks one *backend* process to serve each client. All backends, together with a set of *background* processes (the checkpointer, background writer, WAL writer, and autovacuum, among others), share one region of *shared memory* — most importantly **shared_buffers**, the in-memory cache of table and index pages, and the WAL buffers. That shared memory is persisted to *disk*: the data files and the write-ahead log (WAL).

Three consequences worth stating now, each expanded in its own chapter:

- **One process per connection** makes connections relatively expensive — the motivation for *connection pooling* (its own chapter).
- **Pages live in shared_buffers** before and after disk, so much of performance is about cache behaviour and the *buffer manager*.

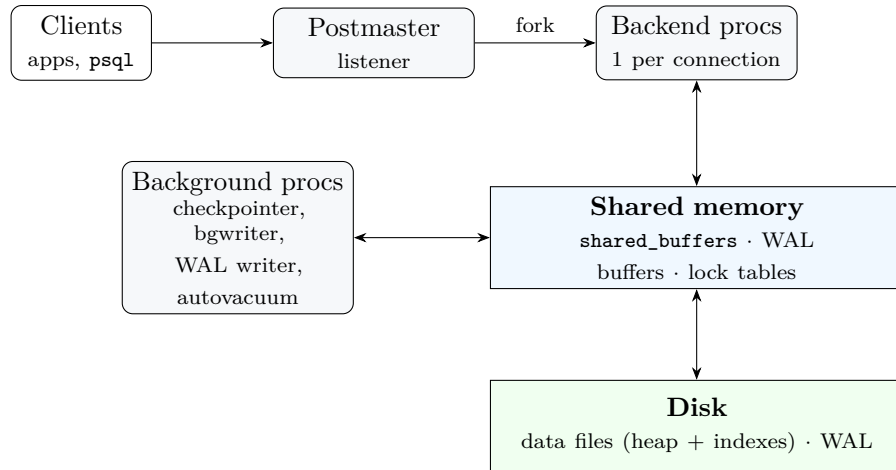


Figure 1: PostgreSQL’s process-and-memory architecture. The *postmaster* listens and forks one *backend* process per client connection. Backends and the *background* processes communicate through a region of *shared memory* (notably **shared_buffers**, the page cache, and the WAL buffers), which is flushed to *disk* — the data files and the write-ahead log. Most later chapters zoom into one box of this picture.

- **Durability rides on the WAL:** changes are written to the log before the data files, which underpins crash recovery and *replication*.

1.3 The life of a query

Within a backend, a SQL statement flows through four stages (fig. 2): the *parser* builds a syntax tree, the *rewriter* applies view and rule expansions, the *planner/optimizer* chooses an execution plan by estimating costs, and the *executor* runs that plan. The parser and rewriter are cheap; the planner and executor are where the interesting behaviour lives, and where the querying chapters focus.

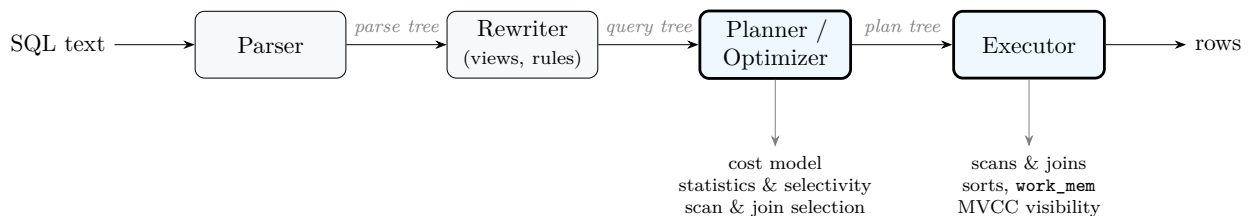


Figure 2: The life of a query. The **Planner** turns a query tree into a *plan tree* by estimating the cost of alternatives; the **Executor** runs the chosen plan. Almost all tuning targets these two stages (highlighted): you change the inputs the planner sees (statistics, indexes, configuration) or the work the executor does (scan/join methods, sorts), then measure again.

1.4 Roadmap

The guide is organized into themes; chapters within a theme are largely independent, so you can read (or extend) in whatever order suits you.

Theme	Topics (each its own chapter)
Foundations	architecture (this chapter); on-disk storage & page layout; MVCC & transactions; WAL & durability
Querying	the planner & EXPLAIN ; indexes; scan methods; joins ; aggregation & sorting
Data modeling	schema design & constraints; partitioning ; data types & TOAST
Scaling & ops	connection pooling ; replication & high availability; vacuum & maintenance
Performance	server configuration & memory; query tuning; monitoring & diagnostics

This list is a starting frame, not a fixed syllabus — topics can be added, split, or reordered. Bold entries (**joins**, **partitioning**, **connection pooling**) are ones already on the radar.

Next: pick any chapter. A natural early one is *on-disk storage and the page layout* (how a row physically lives in a file), since MVCC, indexing, and vacuum all build on it — but the order is yours.

2 Joins

A join combines rows from two relations. PostgreSQL supports the full SQL join vocabulary plus a few semantics (semi, anti, lateral) that you express indirectly. This chapter covers all of them against one small dataset, then shows the three *algorithms* the executor uses to actually compute a join.

2.1 Two orthogonal axes (flag this first)

“Type of join” means two different things, and conflating them causes endless confusion:

- **Logical join type** — *what* rows come out: **CROSS**, **INNER**, **LEFT/RIGHT/FULL OUTER**, plus semi, anti, self, lateral, and non-equi joins. This is SQL semantics.
- **Physical join algorithm** — *how* the executor computes it: *nested loop*, *hash join*, or *merge join*.

These axes are independent: the *same* **LEFT JOIN** may run as a nested loop today and a hash join tomorrow, depending on data sizes and indexes. **EXPLAIN** shows the physical choice; the SQL shows the logical one. We do the logical types first (§2.2–§2.9), then the algorithms (§2.10).

2.2 The running dataset

Every example below uses these three tables. They are deliberately tiny and contain *unmatched rows on both sides* so that outer/semi/anti joins have something to show: employee *Dave* has no department, and department *Marketing* has no employees.

```

1 CREATE TABLE dept (
2   dept_id   int PRIMARY KEY,
3   dept_name text NOT NULL
4 );
5 CREATE TABLE emp (

```

```

6  emp_id      int    PRIMARY KEY,
7  emp_name    text   NOT NULL,
8  dept_id     int    REFERENCES dept,    -- nullable: an employee may have no dept
9  manager_id  int    REFERENCES emp,     -- self-reference: who they report to
10 salary      numeric NOT NULL
11 );
12 CREATE TABLE salary_grade (
13   grade      char(1) PRIMARY KEY,
14   min_salary numeric NOT NULL,
15   max_salary numeric NOT NULL
16 );
17
18 INSERT INTO dept VALUES
19   (10, 'Engineering'), (20, 'Sales'), (30, 'Marketing');    -- 30 has no employees
20 INSERT INTO emp VALUES
21   (1, 'Alice', 10, NULL, 90000),
22   (2, 'Bob',   10, 1,    60000),
23   (3, 'Carol', 20, 1,    50000),
24   (4, 'Dave',  NULL, 2,   75000);    -- Dave has no dept
25 INSERT INTO salary_grade VALUES
26   ('A', 0, 50000), ('B', 50001, 80000), ('C', 80001, 999999);

```

dept		emp				
dept_id	dept_name	emp_id	emp_name	dept_id	manager_id	salary
10	Engineering	1	Alice	10	–	90000
20	Sales	2	Bob	10	1	60000
30	Marketing	3	Carol	20	1	50000
		4	Dave	–	2	75000

2.3 CROSS JOIN — every pairing

The Cartesian product: each `emp` row paired with each `dept` row, no condition (fig. 3).

```

1  SELECT e.emp_name, d.dept_name FROM emp e CROSS JOIN dept d;    -- 4 x 3 = 12
   rows

```

Every other join below is this grid *filtered* by a match condition — and, for outer joins, padded back with NULLs.

2.4 INNER JOIN — matches only

Keeps pairs where the condition holds. Dave (no dept) and Marketing (no emp) both disappear. Figure 4 (top-left) is the picture.

```

1  SELECT e.emp_name, d.dept_name
2  FROM emp e JOIN dept d ON e.dept_id = d.dept_id;    -- "INNER" is the default

```

emp_name	dept_name
Alice	Engineering
Bob	Engineering
Carol	Sales

	Eng	Sales	Mktg
Alice	Alice, Eng	Alice, Sales	Alice, Mktg
Bob	Bob, Eng	Bob, Sales	Bob, Mktg
Carol	Carol, Eng	Carol, Sales	Carol, Mktg
Dave	Dave, Eng	Dave, Sales	Dave, Mktg

Figure 3: **CROSS JOIN** forms every pairing: $|\text{emp}| \times |\text{dept}| = 4 \times 3 = 12$ rows, with no **ON** condition. Every other join type is this grid *filtered* by a match condition (and, for outer joins, padded with **NULLs** for the rows that would otherwise vanish).

Three ways to write the same match. When the key columns share a name you can shorten **ON**:

```

1 ... emp e JOIN dept d ON e.dept_id = d.dept_id      -- explicit
2 ... emp JOIN dept USING (dept_id)                  -- merges the shared column
3 ... emp NATURAL JOIN dept                          -- joins on ALL same-named
    columns

```

Flag: **NATURAL JOIN** silently uses *every* identically-named column; add a column named **created_at** to both tables and the join condition changes under you. Prefer explicit **ON/USING**.

2.5 LEFT, RIGHT, FULL OUTER — keep the unmatched

Outer joins keep unmatched rows from one or both sides, filling the missing side with **NULL** (fig. 4, top-right and bottom-left).

```

1 SELECT e.emp_name, d.dept_name FROM emp e LEFT JOIN dept d ON e.dept_id = d.
   dept_id;
2 SELECT e.emp_name, d.dept_name FROM emp e RIGHT JOIN dept d ON e.dept_id = d.
   dept_id;
3 SELECT e.emp_name, d.dept_name FROM emp e FULL JOIN dept d ON e.dept_id = d.
   dept_id;

```

<u>LEFT</u>		<u>RIGHT</u>		<u>FULL</u>	
emp	dept	emp	dept	emp	dept
Alice	Engineering	Alice	Engineering	Alice	Engineering
Bob	Engineering	Bob	Engineering	Bob	Engineering
Carol	Sales	Carol	Sales	Carol	Sales
Dave	<i>NULL</i>	<i>NULL</i>	Marketing	Dave	<i>NULL</i>
				<i>NULL</i>	Marketing

$A \text{ LEFT JOIN } B$ equals $B \text{ RIGHT JOIN } A$ — **RIGHT** is just **LEFT** with the tables swapped, so most code sticks to **LEFT**.

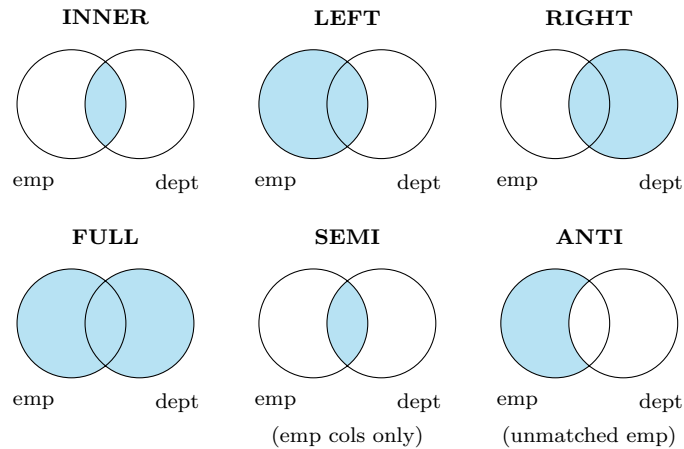


Figure 4: The set-style joins on `emp` (left) and `dept` (right), matching on `emp.dept_id = dept.dept_id`. Shaded = rows returned. INNER keeps only matches; LEFT/RIGHT keep all of one side; FULL keeps everything; SEMI returns matching *left* rows once (no right columns, no duplication); ANTI returns left rows with *no* match.

2.6 SEMI and ANTI joins — filter, don’t combine

A *semi* join returns left rows that *have* a match (once, with no right columns); an *anti* join returns left rows with *no* match (fig. 4, bottom row). PostgreSQL has no SEMI/ANTI keyword — you write EXISTS/NOT EXISTS (or IN), and the planner labels the node “Semi Join”/“Anti Join”.

```

1  -- SEMI: employees who belong to some department -> Alice, Bob, Carol
2  SELECT emp_name FROM emp e WHERE EXISTS (SELECT 1 FROM dept d WHERE d.dept_id
    = e.dept_id);
3  -- ANTI: departments with no employees -> Marketing
4  SELECT dept_name FROM dept d WHERE NOT EXISTS (SELECT 1 FROM emp e WHERE e.
    dept_id = d.dept_id);

```

The NOT IN + NULL footgun. It is tempting to write the anti-join as NOT IN:

```

1  SELECT dept_name FROM dept WHERE dept_id NOT IN (SELECT dept_id FROM emp); --
    returns NOTHING

```

Because `emp.dept_id` contains a NULL (Dave), every NOT IN comparison evaluates to *unknown*, so *no* rows qualify — Marketing is silently lost. NOT EXISTS (or LEFT JOIN ...WHERE right IS NULL) is NULL-safe; prefer it for anti-joins.

2.7 SELF join — a table joined to itself

Not a separate join type, just both sides being the same table (here via `manager_id`). Alias the table twice.

```

1  SELECT e.emp_name AS employee, m.emp_name AS manager
2  FROM emp e JOIN emp m ON e.manager_id = m.emp_id;

```

employee	manager
Bob	Alice
Carol	Alice
Dave	Bob

Alice is dropped (her `manager_id` is NULL); a LEFT self join would keep her with a NULL manager.

2.8 LATERAL join — the right side sees the left

A LATERAL subquery may reference columns of the preceding table, so it runs *per outer row* — ideal for “top-*N* per group”. Here, the first employee of each department:

```

1 SELECT d.dept_name, e.emp_name
2 FROM dept d
3 CROSS JOIN LATERAL (
4     SELECT emp_name FROM emp WHERE emp.dept_id = d.dept_id  -- references d!
5     ORDER BY emp_id LIMIT 1
6 ) e;
```

dept_name	emp_name
Engineering	Alice
Sales	Carol

Marketing is dropped because its lateral subquery returns no rows — exactly like an inner join. Use LEFT JOIN LATERAL (...) e ON true to keep it with a NULL emp_name.

2.9 Non-equi (theta) and range joins

The ON condition need not be equality. Bucketing each salary into its grade is a *range join* (BETWEEN):

```

1 SELECT e.emp_name, e.salary, g.grade
2 FROM emp e JOIN salary_grade g ON e.salary BETWEEN g.min_salary AND g.
   max_salary;
```

emp_name	salary	grade
Alice	90000	C
Bob	60000	B
Carol	50000	A
Dave	75000	B

Non-equi joins cannot use a hash join (hashing needs equality); the planner falls back to nested loop or merge join — a useful thing to recognize in EXPLAIN.

2.10 Physical execution: nested loop, hash, merge

Whatever the logical type, the executor picks one of three algorithms (fig. 5), and the planner chooses by estimated cost. EXPLAIN reveals the choice. The same inner join might appear as either of:

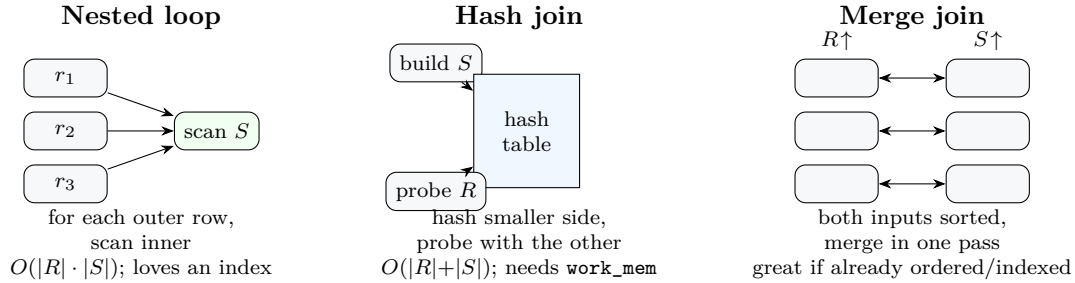


Figure 5: The three ways PostgreSQL physically executes *any* logical join. **Nested loop**: for every outer row, look up matches in the inner — cheap when the inner has a usable index or is tiny. **Hash join**: build a hash table on the smaller input, then probe it with the larger — the workhorse for big unsorted equi-joins. **Merge join**: requires both inputs sorted on the join key, then zips them in a single pass — ideal when the order is free (e.g. from an index). The arrow $\uparrow$ marks a sorted input.

```

1  -- small tables / indexed inner:
2  Nested Loop
3    -> Seq Scan on emp e
4    -> Index Scan using dept_pkey on dept d (cond: d.dept_id = e.dept_id)
5
6  -- larger unsorted inputs, equality key:
7  Hash Join (hash cond: e.dept_id = d.dept_id)
8    -> Seq Scan on emp e
9    -> Hash -> Seq Scan on dept d

```

Rules of thumb: **nested loop** wins when one side is tiny or the inner has an index on the join key; **hash join** wins for large unsorted *equi*-joins (and needs `work_mem` for the hash table); **merge join** wins when both inputs already arrive sorted on the key (e.g. from an index or a prior sort).

2.11 Summary: all the join types

Logical join	How to write it	Returns
cross	CROSS JOIN	every pairing ($ R \times S $)
inner	[INNER] JOIN ...ON	matching pairs only
left outer	LEFT JOIN ...ON	all left, NULL-padded right
right outer	RIGHT JOIN ...ON	all right, NULL-padded left
full outer	FULL JOIN ...ON	all rows, padded both ways
self	JOIN same table (aliased)	rows related within one table
semi	WHERE EXISTS (...)	left rows that have a match
anti	WHERE NOT EXISTS (...)	left rows with no match
lateral	[CROSS LEFT] JOIN LATERAL (...)	per-row correlated subquery
non-equi	JOIN ...ON <, BETWEEN, ...	pairs satisfying a non-equality
Physical algorithm	Plan node	Best when
nested loop	Nested Loop	one side tiny, or indexed inner
hash join	Hash Join	large unsorted equi-join
merge join	Merge Join	both inputs sorted on the key

3 Connection pooling

PostgreSQL runs *one operating-system process per connection* (fig. 1). That single fact makes connections relatively heavy, and it is the entire reason connection pooling exists. This chapter explains why, what a pooler does, the three pooling modes and their trade-offs, how to size a pool, and a concrete PgBouncer setup.

3.1 Why pooling: one backend per connection

Each client connection costs:

- **Setup** — the postmaster forks a backend, authenticates, and warms up catalog caches. For short web requests this handshake can rival the query.
- **Memory** — every backend holds its own caches and can allocate `work_mem` *per operation*; thousands of backends multiply this.
- **Contention** — beyond a modest number of *active* backends, they fight over CPU, locks, and memory bandwidth, so total throughput *drops* (fig. 6).

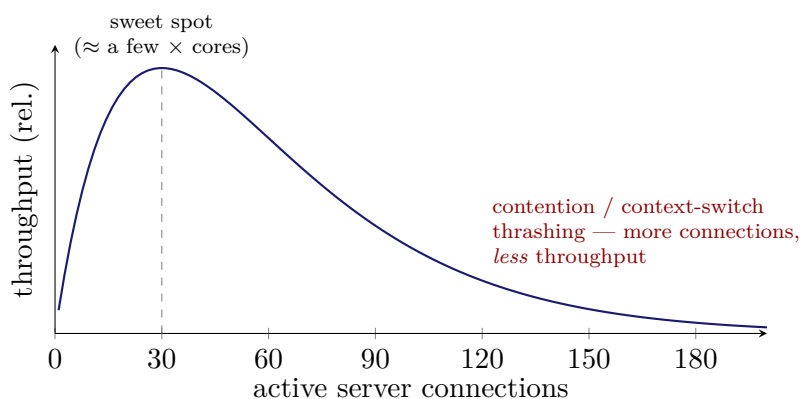


Figure 6: The counterintuitive shape of throughput vs. connection count. Past a modest peak — a small multiple of the CPU core count — adding connections *lowers* throughput, because backends contend for CPU, locks, and memory bandwidth. This is why a pooler that *caps* server connections (and queues the rest) usually beats letting every client connect directly. The curve is schematic; the real peak is found by measuring on your own workload.

The tension: applications want many connections (one per thread/request), while the server is fastest with few active ones. A pooler resolves it by *multiplexing* many client connections onto a small, fixed set of server connections (fig. 7).

3.2 Where pooling lives

- **In-app pool** (e.g. HikariCP, `psycpg_pool`) — reuses connections within one app instance. Necessary, but each instance pools independently, so N instances $\times$ pool size can still flood the server.
- **External pooler** (PgBouncer, Odyssey, pgcat) — a separate process between apps and PostgreSQL that enforces *one global* cap on server connections. PgBouncer is the de-facto standard and the focus here.

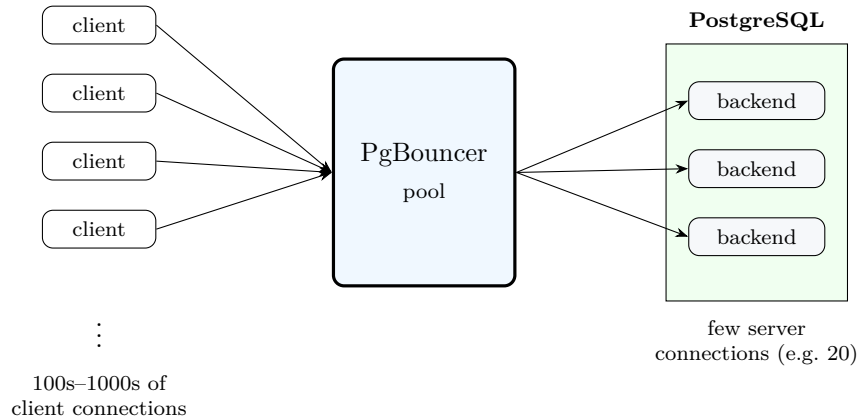


Figure 7: A connection pooler sits between the application and PostgreSQL and *multiplexes*: it accepts a very large number of (mostly idle) client connections and serves them from a small, fixed set of real server connections — i.e. a small set of backends (fig. 1). This both caps the per-backend memory and context-switch overhead and avoids paying connection-setup cost on every request.

Note: PostgreSQL core ships *no* built-in pooler (through PG 16/17), so an external pooler is a deliberate, separate component.

3.3 The three pooling modes

A pooler differs from a plain proxy in *when* it returns a server connection to the pool. That choice is `pool_mode` (fig. 8).

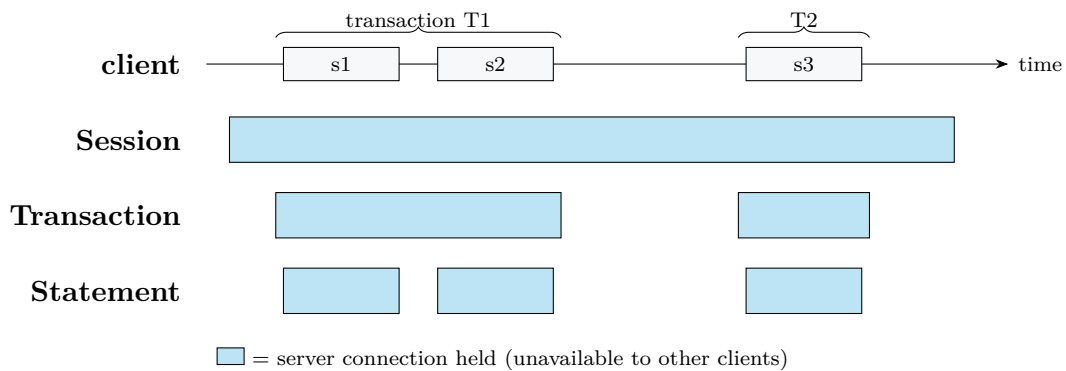


Figure 8: PgBouncer pooling modes, for one client running two transactions (T1 with statements s1,s2; T2 with s3). **Session:** the server connection is bound for the client’s whole session — safe but little sharing. **Transaction:** bound only during each transaction and returned at COMMIT/ROLLBACK, so it is free during the gap for *other* clients — maximum practical multiplexing. **Statement:** released after every statement; forbids multi-statement transactions. Shorter held intervals $\Rightarrow$ more clients per server connection, but fewer session features survive.

Mode	Server conn returned	Trade-off
session	when the client disconnects	safest — all session features work; least multiplexing
transaction	at each COMMIT/ROLLBACK	the common choice for web apps; breaks cross-transaction session state
statement	after every statement	maximum sharing; forbids multi-statement transactions

3.4 What transaction pooling breaks (and how to cope)

In `transaction` mode a client gets a *different* backend per transaction, so anything that lives in `session` state across transactions is unsafe:

- **Session-level SET** (e.g. `SET search_path`, `SET TIME ZONE`) — use `SET LOCAL` inside the transaction, or set it in the connection’s startup parameters.
- **Server-side prepared statements** — a prepare on one backend is not visible on another; either disable them in the driver or use a PgBouncer version with prepared-statement support enabled.
- **Cross-transaction features** — `LISTEN/NOTIFY`, advisory locks held across transactions, `WITH HOLD` cursors, session-scoped temp tables. Keep these on a `session`-mode pool (or a direct connection).

PgBouncer runs `server_reset_query = DISCARD ALL` between uses to scrub leftover state; it is required for `session` mode and a safety net generally.

3.5 Sizing the pool

Bigger is not better (fig. 6). A widely-used starting heuristic for the number of *server* connections is

$$\text{pool_size} \approx (\text{CPU cores} \times 2) + \text{effective_spindle_count}, \quad (1)$$

where *effective_spindle_count* approximates how many I/O requests the storage can usefully service in parallel (for SSD/cloud, treat it as a small constant and lean on the $\times 2$ term). Treat the result as a starting point and find the true peak by measurement. The client-facing cap (`max_client_conn`) can be far larger, since idle clients cost the pooler almost nothing.

3.6 A concrete PgBouncer setup

```

1 ; pgbouncer.ini
2 [databases]
3 appdb = host=10.0.0.5 port=5432 dbname=appdb
4
5 [pgbouncer]
6 listen_addr      = *
7 listen_port      = 6432
8 auth_type        = scram-sha-256
9 auth_file        = /etc/pgbouncer/userlist.txt
10
11 pool_mode        = transaction ; session | transaction | statement

```

```

12 max_client_conn      = 2000          ; client connections the pooler accepts
13 default_pool_size    = 20            ; server connections per (user, database)
14 reserve_pool_size    = 5             ; extra conns for bursts
15 server_reset_query    = DISCARD ALL   ; scrub session state between uses

```

The application then connects to PgBouncer’s port instead of PostgreSQL’s — usually the only application change required:

```

1 # was: postgresql://app@db-host:5432/appdb
2 postgresql://app@pgbouncer-host:6432/appdb

```

Here up to 2000 clients share 20 server connections per database — a 100:1 multiplexing ratio that keeps the server in the flat part of fig. 6.

3.7 Choosing a mode

Workload	Recommended mode
Stateless web/API (short transactions)	transaction — highest density; avoid session-scoped state
Needs prepared stmts / temp tables / LISTEN	session (or a direct connection for those paths)
Many parallel one-shot statements, no multi-statement transactions	statement (rare)

Takeaway. Pool because backends are processes; pick **transaction** mode for typical web workloads; cap server connections to a small multiple of cores and verify by measurement; and keep session-stateful code paths off the transaction pool.

4 The psql client

psql is PostgreSQL’s interactive terminal — the tool you live in for ad-hoc queries, inspection, scripting, and bulk loading. It does two distinct things: it sends *SQL* to the server, and it runs *meta-commands* (backslash commands) locally. Examples here reuse the **emp/dept** tables from section 2.

4.1 Connecting

Three interchangeable ways to say where and who:

```

1 psql -h db.example.com -p 5432 -U app -d appdb # flags
2 psql "postgresql://app@db.example.com:5432/appdb" # connection URI
3 PGHOST=db.example.com PGUSER=app PGDATABASE=appdb psql # environment variables

```

Passwords are best kept out of the command line: put them in `~/.pgpass` (host:port:db:user:password, `chmod 600`) or use a named connection in `~/.pg_service.conf` and connect with `psql service=appdb`.

4.2 Two kinds of input: SQL vs. meta-commands

This is the one idea that makes psql make sense (fig. 9): a line starting with `\` is a meta-command handled by psql; anything else is SQL buffered until a `;`.

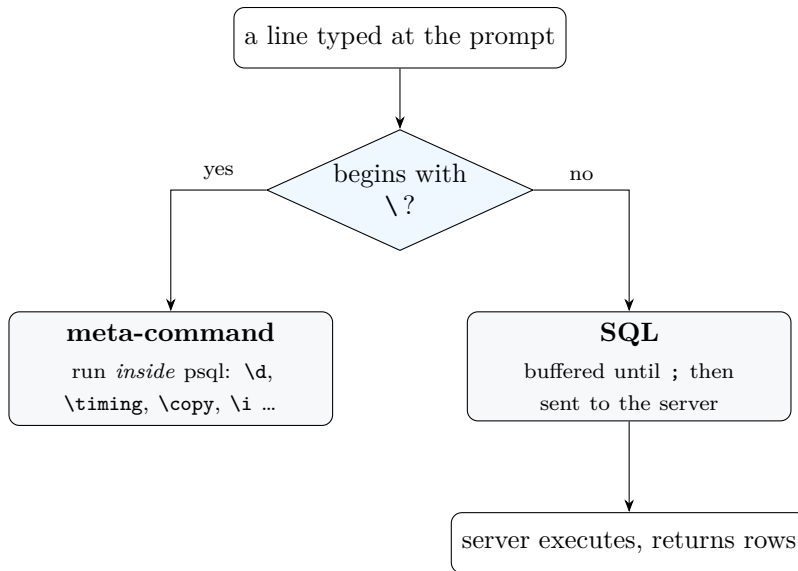
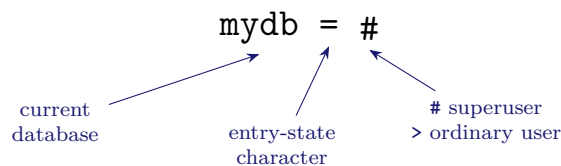


Figure 9: How psql dispatches each line. A leading backslash means a *meta-command*, executed by psql itself and never sent to the server; everything else is *SQL*, accumulated in the query buffer until a semicolon (or \g) ships it. That split is why \d is instant while a **SELECT** makes a round trip.

4.3 Reading the prompt

The prompt encodes state (fig. 10). The middle character is the usual culprit when “nothing happens” after you hit Enter — a - means psql is still waiting for a semicolon.



entry-state character: = ready - need more input (no ; yet) (open parenthesis ' inside a string * inside a */**/* comment

Figure 10: Anatomy of the default prompt `mydb=#`: database name, then the *entry-state* character, then # (superuser) or > (ordinary user). The middle character is how you answer “why hasn’t my statement run?” — usually a missing semicolon (-) or an unclosed quote (').

4.4 Inspecting the database: the \d family

The backslash-d commands introspect the catalog without you writing catalog queries. Add + for more detail (sizes, descriptions); add S to include system objects.

<code>\l</code>	list databases	<code>\dn</code>	schemas
<code>\dt</code>	tables	<code>\dv</code>	views
<code>\di</code>	indexes	<code>\df</code>	functions
<code>\d <i>name</i></code>	describe a table/view/index	<code>\du</code>	roles (users)
<code>\d+ <i>name</i></code>	describe, with extra detail	<code>\dx</code>	installed extensions
<code>\sf <i>func</i></code>	show a function's source	<code>\dp</code>	table privileges

For example, `\d emp` prints the columns, indexes, and foreign keys:

```

1 mydb=# \d emp
2
3           Table "public.emp"
4  Column | Type | Nullable | Default
5 -----+-----+-----+-----
6 emp_id | integer | not null |
7 emp_name | text | not null |
8 dept_id | integer | |
9 manager_id | integer | |
10 salary | numeric | not null |
11
12 Indexes:
13     "emp_pkey" PRIMARY KEY, btree (emp_id)
14 Foreign-key constraints:
15     "emp_dept_id_fkey" FOREIGN KEY (dept_id) REFERENCES dept(dept_id)
16     "emp_manager_id_fkey" FOREIGN KEY (manager_id) REFERENCES emp(emp_id)

```

4.5 Output formatting: aligned, expanded, `\pset`

Wide rows are unreadable in the default aligned grid; `\x` switches to *expanded* (one field per line). `\x` auto picks per result.

```

1 mydb=# SELECT * FROM emp WHERE emp_id = 1;
2  emp_id | emp_name | dept_id | manager_id | salary
3  -----+-----+-----+-----+-----
4      1 | Alice | 10 | | 90000
5
6 mydb=# \x
7 Expanded display is on.
8 mydb=# SELECT * FROM emp WHERE emp_id = 1;
9  -[ RECORD 1 ]----
10 emp_id | 1
11 emp_name | Alice
12 dept_id | 10
13 manager_id |
14 salary | 90000

```

Other knobs via `\pset`: `\pset null '(null)'` (how NULL prints), `\pset format csv`, `\t` (rows only, no header/footer), `\a` (unaligned).

4.6 The query buffer: `\e`, `\i`, `\g`

psql keeps the last SQL in a buffer. `\e` opens it in `$EDITOR`; `\g` re-runs it (optionally to a file: `\g out.txt`); `\i file.sql` runs a script; `\o file` redirects subsequent query output to a file.

4.7 Variables and interpolation

`\set` defines a variable; `:-` prefixes interpolate it. The quoting form matters — this is a common SQL-injection footgun if you forget it:

```
1 mydb=# \set id 1
2 mydb=# SELECT emp_name FROM emp WHERE emp_id = :id; -- bare: numeric/raw
3 mydb=# \set who Alice
4 mydb=# SELECT * FROM emp WHERE emp_name = :'who'; -- :'x' -> literal 'Alice'
5 mydb=# \set col emp_name
6 mydb=# SELECT :col FROM emp; -- :x -> identifier "emp_name"
7 mydb=# SELECT count(*) AS n FROM emp \gset -- capture result into :n
8 mydb=# \echo :n
9 4
```

4.8 Importing and exporting: `\copy` vs. `COPY`

A crucial distinction covered in depth next chapter: `COPY` reads/writes a file on the *server* (and needs file privileges), while `\copy` is a psql meta-command that streams through the *client*, so the path is on *your* machine.

```
1 mydb=# \copy emp TO 'emp.csv' WITH (FORMAT csv, HEADER) -- writes ./emp.csv locally
2 mydb=# \copy emp FROM 'emp.csv' WITH (FORMAT csv, HEADER) -- loads from ./emp.csv
```

4.9 Measuring and watching

`\timing` on reports wall-clock per statement; `\watch N` re-runs the last query every N seconds (a poor person's monitor).

```
1 mydb=# \timing on
2 mydb=# SELECT count(*) FROM emp;
3 count
4 -----
5      4
6 Time: 0.412 ms
7 mydb=# SELECT state, count(*) FROM pg_stat_activity GROUP BY state;
8 mydb=# \watch 5 -- repeat every 5 seconds; Ctrl-C to stop
```

4.10 Scripting psql non-interactively

For migrations and CI, drive psql from the shell. `ON_ERROR_STOP=1` is essential — without it psql plows past failed statements and still exits 0.

```
1 psql -d appdb -v ON_ERROR_STOP=1 -f migrate.sql # run a script, fail fast
2 psql -d appdb -c "SELECT count(*) FROM emp" # one statement
3 psql -d appdb -Atc "SELECT emp_name FROM emp" # -A unaligned, -t tuples-only:
4                                           # clean output for pipes
```

`-v name=value` sets a variable a script reads as `:name`; the exit code is non-zero on error (with `ON_ERROR_STOP`), which is what CI checks.

4.11 Customizing with .psqlrc

~/.psqlrc runs at startup. A practical starting point:

```
1 \set QUIET 1
2 \timing on
3 \x auto
4 \pset null '(null)'
5 \set COMP_KEYWORD_CASE upper
6 \set HISTSIZE 100000
7 \set PROMPT1 '%n%/%R%# ' -- user@db, state, #/>
8 \set QUIET 0
```

4.12 Meta-command quick reference

\?	help on meta-commands	\h <i>SQL</i>	help on SQL syntax
\c <i>db</i>	connect to another database	\conninfo	current connection info
\x	toggle expanded output	\timing	toggle per-statement timing
\e	edit buffer in \$EDITOR	\i <i>file</i>	run a SQL script
\g	re-run buffer	\o <i>file</i>	send output to a file
\copy	client-side import/export	\watch <i>N</i>	repeat query every <i>N</i> s
\set / \gset	set / capture variables	\q	quit

5 INSERT and COPY

Two ways to put rows into a table: **INSERT** (flexible, row- or query-oriented, with upsert) and **COPY** (a bulk stream, far faster for large loads). This chapter covers both in full and shows when each wins. Examples reuse **emp** from section 2.

5.1 INSERT: the basic forms

```
1 -- 1) single row, naming columns (recommended: order-independent, future-proof
  )
2 INSERT INTO emp (emp_id, emp_name, dept_id, manager_id, salary)
3 VALUES (5, 'Erin', 20, 3, 55000);
4
5 -- 2) multiple rows in one statement (one round trip; see fig.)
6 INSERT INTO emp (emp_id, emp_name, dept_id, manager_id, salary) VALUES
7 (6, 'Frank', 10, 1, 62000),
8 (7, 'Grace', 20, 3, 58000);
9
10 -- 3) insert the result of a query
11 INSERT INTO emp (emp_id, emp_name, dept_id, salary)
12 SELECT 100 + g, 'temp' || g, 10, 40000
13 FROM generate_series(1, 5) AS g; -- inserts 5 generated rows
```

Omitted columns take their **DEFAULT** (or **NULL**); write **DEFAULT** explicitly to be unambiguous. For **GENERATED/identity** or **serial** keys, leave the column out and let the sequence fill it.

5.2 RETURNING: get data back from the write

A PostgreSQL extension: INSERT/UPDATE/DELETE can return rows, which avoids a follow-up SELECT (essential for server-generated keys).

```
1 INSERT INTO emp (emp_name, dept_id, salary)
2 VALUES ('Heidi', 10, 70000)
3 RETURNING emp_id, emp_name;      -- returns the new emp_id assigned by the
    sequence
```

5.3 Upsert: INSERT ...ON CONFLICT

“Insert, or update if it already exists” is one atomic statement (fig. 11). You name the *conflict target* (a unique constraint/index) and an action.

```
1 -- insert-or-ignore: do nothing if emp_id 1 already exists
2 INSERT INTO emp (emp_id, emp_name, salary) VALUES (1, 'Alice', 0)
3 ON CONFLICT (emp_id) DO NOTHING;
4
5 -- insert-or-update: bump salary if the row exists; EXCLUDED = the row we
    tried to insert
6 INSERT INTO emp (emp_id, emp_name, salary) VALUES (1, 'Alice', 95000)
7 ON CONFLICT (emp_id) DO UPDATE
8   SET salary = EXCLUDED.salary
9   WHERE emp.salary < EXCLUDED.salary;      -- optional guard: only raise, never
    lower
```

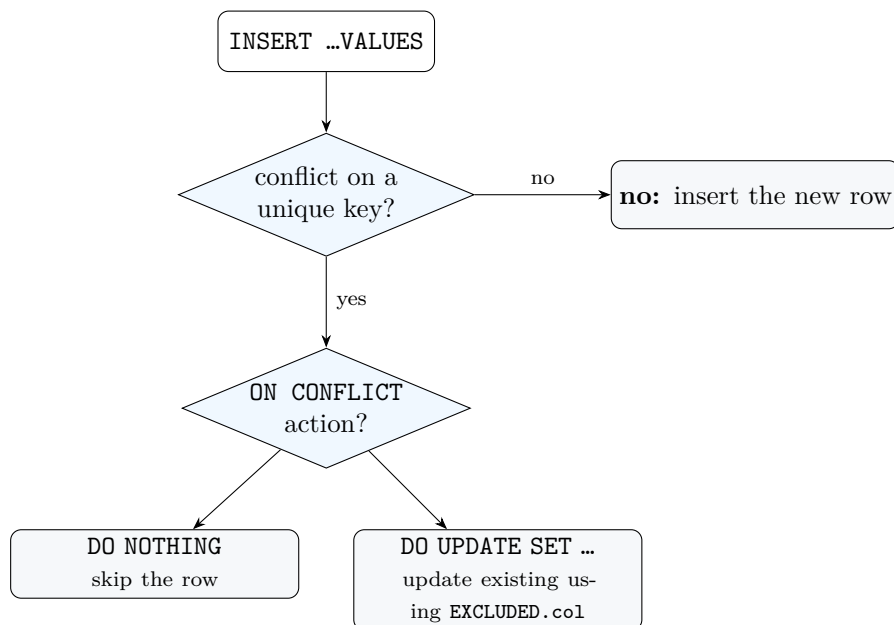


Figure 11: INSERT ...ON CONFLICT (upsert). If the new row would violate a unique constraint/index named by the conflict target, PostgreSQL takes the ON CONFLICT action instead of erroring: DO NOTHING silently skips it, or DO UPDATE edits the existing row, where the special EXCLUDED row holds the values you tried to insert.

Flags: the conflict target must be a real unique constraint or unique index — `ON CONFLICT` cannot fire on an arbitrary column. Inside `DO UPDATE`, `EXCLUDED.col` is the proposed value and the bare table name is the existing row.

5.4 COPY: bulk load and unload

`COPY` moves many rows in a single command, parsed once (fig. 13) — the right tool for thousands-plus rows.

```
1 -- unload a table or a query to CSV (server-side path)
2 COPY emp TO '/tmp/emp.csv' WITH (FORMAT csv, HEADER);
3 COPY (SELECT emp_name, salary FROM emp WHERE salary > 60000)
4     TO '/tmp/highpaid.csv' WITH (FORMAT csv, HEADER);
5
6 -- load from CSV
7 COPY emp FROM '/tmp/emp.csv' WITH (FORMAT csv, HEADER);
```

Common options: `FORMAT` (`text` | `csv` | `binary`), `HEADER`, `DELIMITER` `'`, `NULL` `'` (how `NULL` is spelled), `QUOTE/ESCAPE`, and `FORCE_NOT_NULL`. `FREEZE` can speed a load into a fresh table.

5.5 COPY (server) vs. \copy (client)

The single most common `COPY` confusion (fig. 12): `COPY ...'/path'` reads/writes that path *on the database server* and needs file privileges; `psql`'s `\copy` streams through the client, so the path is local to *you*. Same syntax otherwise.

```
1 -- in psql: file is on your machine, no server file privileges required
2 mydb=# \copy emp FROM 'emp.csv' WITH (FORMAT csv, HEADER)
3
4 -- programmatic streaming uses STDIN/STDOUT (what drivers and \copy use under the hood)
5 COPY emp FROM STDIN WITH (FORMAT csv);
6 5,Erin,20,3,55000
7 \.
```

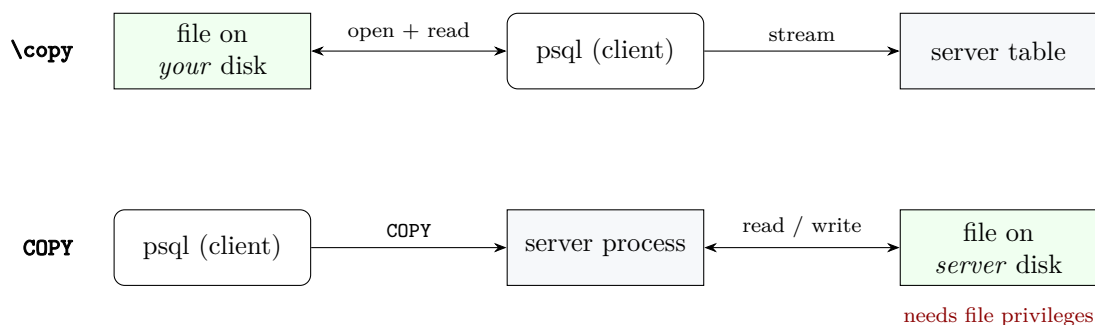


Figure 12: The two bulk paths differ in *where the file is*. `\copy` is a `psql` meta-command: `psql` opens the file on the *client* and streams it over the existing connection — no server file access needed. `COPY` runs on the *server*, reading/writing a path on the server's filesystem, which requires elevated privileges (superuser or `pg_read_server_files/pg_write_server_files`).

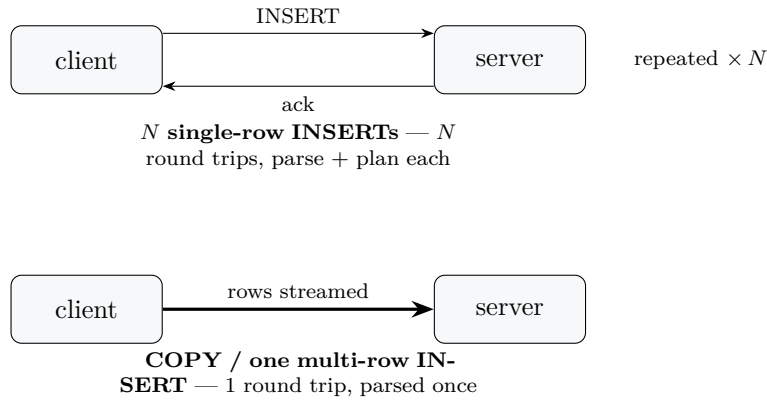


Figure 13: The performance gap. Each standalone `INSERT` is a full round trip that is parsed and planned on its own; loading N rows that way costs N of everything. `COPY` (and, to a lesser degree, a single multi-row `INSERT`) streams all rows in one command, parsed once — typically an order of magnitude or more faster for bulk loads.

5.6 Why `COPY` beats even a multi-row `INSERT`

A single multi-row `INSERT` already removes the per-row *round trip* (fig. 13), so why is `COPY` still faster? Because it also removes the per-row *SQL processing*:

- **Parsing.** A multi-row `INSERT` is still a SQL statement: the parser and analyzer build parse-tree nodes for *every* value expression in the `VALUES` list. `COPY` reads input with a lightweight field scanner (text/CSV/binary) — no SQL grammar per row.
- **Planning & execution.** `INSERT` plans a query and pushes each row through the executor’s expression evaluation. `COPY` feeds rows through a tight ingest loop that skips per-row planning and most of that machinery.
- **Memory & limits.** The whole multi-row statement is parsed at once, so very large `VALUES` lists balloon parser memory; parameterized inserts also hit the wire protocol’s $\sim 65,535$ -parameter ceiling. `COPY` streams with bounded memory and no such limit.
- **WAL.** `COPY` into a table *created or truncated in the same transaction* (with `wal_level=minimal`) can skip WAL for that table, and `COPY ...WITH (FREEZE)` pre-freezes the rows. A multi-row `INSERT` gets neither optimization.

Net: a multi-row `INSERT` closes the *round-trip* gap but not the *per-row* processing gap; `COPY` closes both, so it stays faster — and the margin widens as the row count grows.

5.7 Choosing, and loading fast

Goal	Use
A few rows, or app logic per row	<code>INSERT</code> (multi-row <code>VALUES</code> when you can)
Insert-or-update semantics	<code>INSERT ...ON CONFLICT</code>
Need generated keys back	<code>INSERT ...RETURNING</code>
Thousands+ of rows	<code>COPY</code> (or <code>\copy</code> from the client)

For a big one-off load, beyond using `COPY`: load inside a single transaction; drop and recreate secondary indexes around the load; raise `maintenance_work_mem` for the rebuild; and consider an `UNLOGGED` table (or the `FREEZE` option) when the data can be regenerated. We return to these knobs in the performance chapter.

6 Ordering query results (`ORDER BY`)

How to get rows back in a chosen order — and what it costs. Examples reuse `emp` from section 2 (salaries: Alice 90000, Dave 75000, Bob 60000, Carol 50000; Dave has a `NULL dept_id`).

6.1 Rows have no order until you ask (flag)

A SQL table is a *set*: a query without `ORDER BY` may return rows in *any* order, and that order is not stable — it is whatever the plan produces and can change with data size, statistics, `VACUUM`, parallelism, or a version upgrade (fig. 14). Do not rely on insertion order, primary-key order, or “the order it showed last time”. `ORDER BY` is the only guarantee.

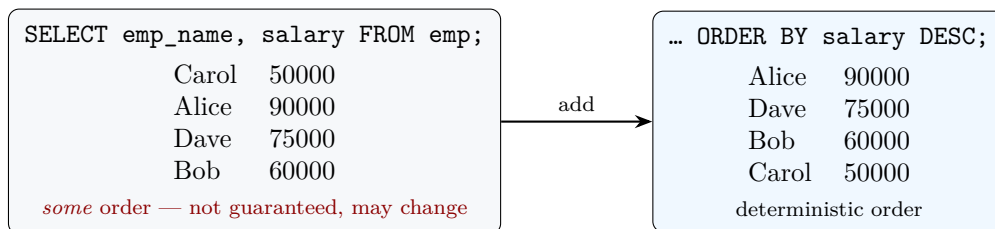


Figure 14: A query result is a *set* until you order it. Without `ORDER BY`, PostgreSQL may return rows in any order — whatever the plan happens to produce — and that order can change with data, statistics, or version. `ORDER BY` is the only way to guarantee a sequence.

6.2 The basics: keys, direction, multiple columns

```
1 SELECT emp_name, salary FROM emp ORDER BY salary DESC;      --
   90000,75000,60000,50000
2 SELECT emp_name FROM emp ORDER BY emp_name;                -- ASC is the
   default
3 SELECT * FROM emp ORDER BY dept_id, salary DESC;           -- group by
   dept, then high-pay first
```

Each key has its own direction (`ASC/DESC`). Later keys break ties from earlier ones, so `ORDER BY dept_id, salary DESC` sorts within each department.

6.3 NULL ordering

`NULL` sorts as “larger than everything”. The defaults follow that: `NULLS LAST` for `ASC`, `NULLS FIRST` for `DESC`. Override explicitly:

```
1 SELECT emp_name, dept_id FROM emp ORDER BY dept_id;         -- Dave (NULL
   dept) last (ASC default)
2 SELECT emp_name, dept_id FROM emp ORDER BY dept_id NULLS FIRST; -- Dave first
   instead
```



```
3 SELECT emp_name, manager_id FROM emp ORDER BY manager_id DESC NULLS LAST;
```

Flag: mixing DESC with default null placement surprises people — spell out NULLS FIRST/LAST whenever nulls matter.

6.4 Ordering by expressions, aliases, positions, collation

```
1 SELECT emp_name, salary*12 AS annual FROM emp ORDER BY annual;      -- by
   output alias
2 SELECT emp_name, salary FROM emp ORDER BY salary*12;                -- by an
   expression
3 SELECT emp_name FROM emp ORDER BY lower(emp_name);                  -- case-
   insensitive-ish
4 SELECT emp_name, salary FROM emp ORDER BY 2 DESC;                   -- by column
   POSITION (2 = salary)
5 SELECT emp_name FROM emp ORDER BY emp_name COLLATE "C";             -- byte
   order, not locale
```

Text ordering depends on the column’s *collation*; COLLATE "C" gives fast, locale-independent byte ordering. ORDER BY <position> is legal but fragile (it breaks when you reorder the SELECT list) — prefer names or aliases.

6.5 Top-*N* and pagination: ORDER BY with LIMIT

ORDER BY plus LIMIT is a top-*N* query, which the planner can answer with a cheap top-*N* heapsort (or an index, below):

```
1 SELECT emp_name, salary FROM emp ORDER BY salary DESC LIMIT 2;      -- Alice,
   Dave
```

LIMIT ...OFFSET paginates, but **deep OFFSET is slow**: the server still computes and discards every skipped row. For large datasets prefer *keyset* (“seek”) pagination, which uses the last row’s key and an index:

```
1 -- page 1
2 SELECT emp_id, salary FROM emp ORDER BY salary DESC, emp_id DESC LIMIT 10;
3 -- next page: continue after the last seen (salary, emp_id)
4 SELECT emp_id, salary FROM emp
5 WHERE (salary, emp_id) < (:last_salary, :last_emp_id)  -- row-value
   comparison
6 ORDER BY salary DESC, emp_id DESC LIMIT 10;
```

Adding emp_id as a tiebreaker makes the order *total* (deterministic across pages even when salaries tie).

6.6 DISTINCT ON: first row per group

A PostgreSQL extension that returns the first row of each group, where “first” is defined by ORDER BY. The leading ORDER BY columns must match the DISTINCT ON list:

```
1 -- highest-paid employee in each department
2 SELECT DISTINCT ON (dept_id) dept_id, emp_name, salary
3 FROM emp
```

```
4 ORDER BY dept_id, salary DESC;      -- dept 10 -> Alice; dept 20 -> Carol; NULL
    dept -> Dave
```

6.7 How ordering executes

Two ways the rows actually become ordered:

- **A Sort node** (fig. 15): in-memory quicksort if the data fits `work_mem`, else an external merge sort that spills to disk. `EXPLAIN (ANALYZE)` prints the `Sort Method` and memory/disk used.
- **An index** (fig. 16): a B-tree on the `ORDER BY` key returns rows pre-sorted, so the plan has *no* Sort node and a `LIMIT` can stop early.

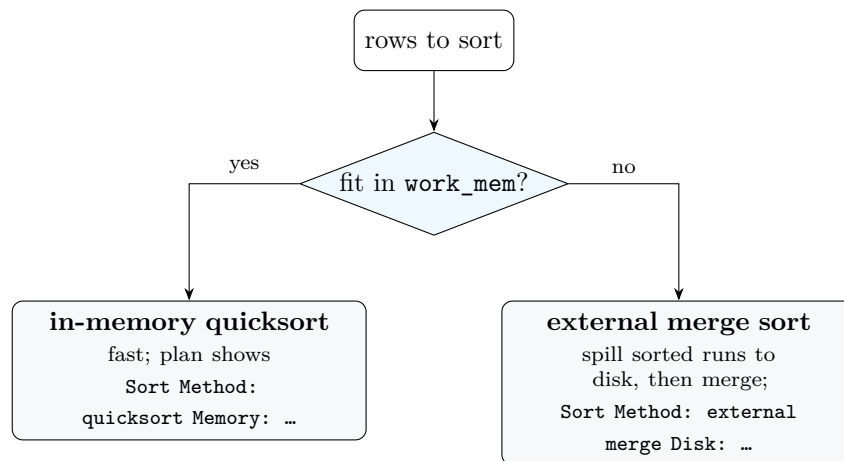


Figure 15: How a `Sort` node runs. If the rows fit in `work_mem` the executor sorts in memory (quicksort, or a top- N heapsort when there is a `LIMIT`); otherwise it spills sorted runs to disk and merges them — an *external merge sort*, much slower. `EXPLAIN (ANALYZE)` prints which path ran, so a `external merge Disk:` line is a signal to raise `work_mem` or avoid the sort (next figure).

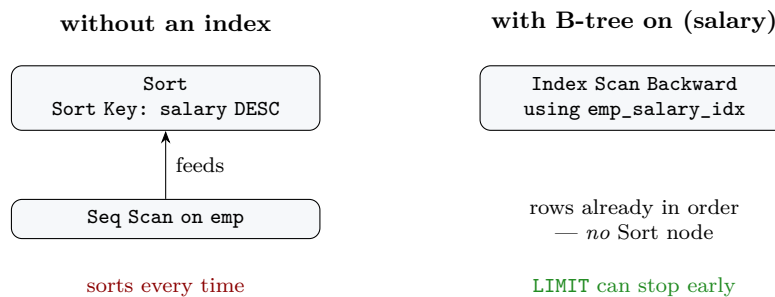


Figure 16: An index on the `ORDER BY` key lets the executor read rows in sorted order directly, eliminating the `Sort` node (and its `work_mem`/spill risk). It also makes `ORDER BY ...LIMIT n` cheap, since the scan can stop after n rows instead of sorting the whole table.

```
1 -- no index: a Sort over a scan
2 mydb=# EXPLAIN ANALYZE SELECT * FROM emp ORDER BY salary DESC;
```

```

3 Sort (cost=1.08..1.09 rows=4 width=...)
4   Sort Key: salary DESC
5   Sort Method: quicksort Memory: 25kB
6   -> Seq Scan on emp
7 -- with CREATE INDEX emp_salary_idx ON emp (salary);
8 Index Scan Backward using emp_salary_idx on emp -- ordered read, no Sort node

```

6.8 Summary

- No **ORDER BY** $\Rightarrow$ no guaranteed order. Add a total order (include a unique tiebreaker) when determinism matters.
- Be explicit about **NULLS FIRST/LAST** and, for text, **COLLATE**.
- For top- N and pagination, pair **ORDER BY** with **LIMIT**; use keyset pagination instead of deep **OFFSET**.
- Watch **EXPLAIN** for external merge **Disk:** (raise **work_mem**) and for a **Sort** that a matching index could remove.

7 The logical order of a SELECT

You *write* a query as **SELECT ...FROM ...WHERE ...**, but the database *evaluates* the clauses in a different, fixed order. Knowing that order is the key that unlocks most “why can’t I do that?” questions in SQL. Examples reuse **emp** from section 2.

7.1 Written order is not evaluation order

Figure 17 puts the two side by side. The headline: **SELECT** is written first but evaluated almost last (fifth of eight). Everything else follows from that.

7.2 The eight phases

Each phase consumes the output of the one before it (fig. 18). A name that a phase creates is therefore visible only to *later* phases.

1. **FROM / JOIN** — the tables are read and joined (section 2) into a single working set of rows. Subqueries in **FROM** and **WITH** CTEs are materialized conceptually here, before anything else runs.
2. **WHERE** — each row is tested and kept or discarded. Because grouping has not happened yet, *aggregates are not allowed*; because **SELECT** has not run yet, *output aliases do not exist*.
3. **GROUP BY** — surviving rows are folded into one row per distinct grouping key.
4. **HAVING** — groups are filtered. Now aggregates *are* allowed, since the groups exist.
5. **SELECT** — the output expressions are computed (including window functions, which run after grouping). This is where column *aliases* are born.
6. **DISTINCT** — duplicate output rows are removed.

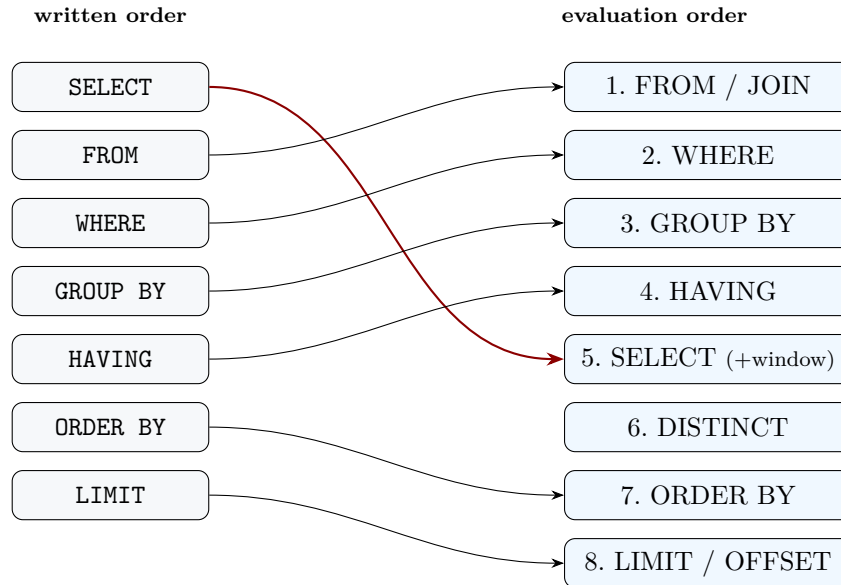


Figure 17: You *write* **SELECT** first, but it is *evaluated* fifth (red). That single mismatch explains most SQL surprises: **WHERE** (step 2) runs before **SELECT** (step 5), so it cannot see **SELECT** aliases; **ORDER BY** (step 7) runs after, so it can. **DISTINCT** (step 6) has no keyword of its own — it is the **DISTINCT** in **SELECT DISTINCT**.

7. **ORDER BY** — the result is sorted. It runs after **SELECT**, so it *can* use aliases and computed columns.

8. **LIMIT / OFFSET** — a slice of the ordered result is returned.

7.3 Why the order matters: the consequences

Every rule below is a direct corollary of the phase order — not an arbitrary quirk.

1. **WHERE** cannot see **SELECT** aliases (alias born in phase 5, used in phase 2).

```

1 SELECT salary * 12 AS annual FROM emp WHERE annual > 700000;    -- ERROR:
   column "annual" does not exist
2 SELECT salary * 12 AS annual FROM emp WHERE salary * 12 > 700000;  -- repeat
   the expression, or use a subquery/CTE

```

2. **WHERE** filters rows; **HAVING** filters groups (phase 2 vs phase 4).

```

1 SELECT dept_id, count(*) AS headcount
2 FROM emp
3 WHERE dept_id IS NOT NULL      -- row filter, BEFORE grouping: no aggregates
   here
4 GROUP BY dept_id
5 HAVING count(*) > 1;          -- group filter, AFTER grouping: aggregates
   allowed
6 -- dept_id / headcount
7 -- -----+-----
8 --      10 /         2

```

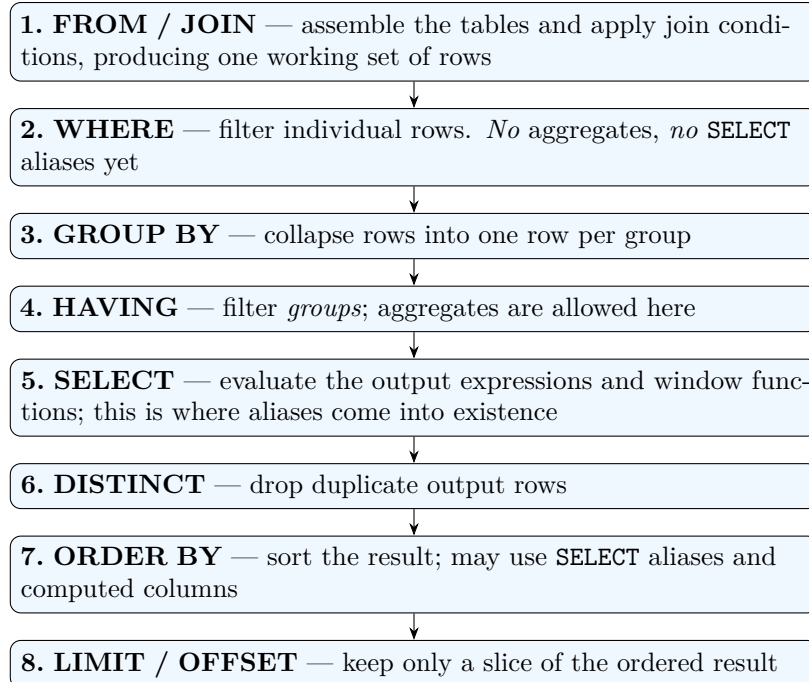


Figure 18: The logical pipeline of a **SELECT**. Each phase consumes the previous phase’s output, which is exactly why a name created in one phase is visible only to *later* phases. This is the *semantic* order; the planner may physically compute things in another order (fig. 2) as long as the answer is identical.

Put a condition in **WHERE** when it concerns a single row (cheaper — fewer rows to group); put it in **HAVING** only when it needs an aggregate.

3. **ORDER BY** can use aliases and **SELECT**-computed columns (phase 7 after 5).

```

1 SELECT emp_name, salary * 12 AS annual FROM emp ORDER BY annual DESC;  --
    alias is fine here

```

4. A non-grouped column in **SELECT** is an error (phase 3 constrains phase 5).

```

1 SELECT dept_id, emp_name FROM emp GROUP BY dept_id;  -- ERROR: emp_name must
    appear in GROUP BY or an aggregate

```

After grouping, a group no longer has a single `emp_name`; you must aggregate it (e.g. `max(emp_name)`) or group by it.

5. Window functions can’t be filtered in **WHERE**/**HAVING** (they run inside phase 5).

```

1 -- ERROR: window functions are not allowed in WHERE
2 SELECT emp_name FROM emp WHERE rank() OVER (ORDER BY salary DESC) <= 2;
3 -- wrap it: compute in an inner SELECT (phase 5), filter in the outer WHERE (
    phase 2 of the outer query)
4 SELECT emp_name FROM (
5     SELECT emp_name, rank() OVER (ORDER BY salary DESC) AS r FROM emp
6 ) ranked
7 WHERE r <= 2;  -- Alice, Dave

```

6. LIMIT applies last; aggregates ignore it (phase 8 is the very end).

```
1 SELECT count(*) FROM emp LIMIT 1;      -- count() sees all 4 rows; LIMIT only
    caps output rows
2 SELECT emp_name FROM emp ORDER BY salary DESC LIMIT 2;  -- the WHOLE set is
    ordered, then sliced
```

7.4 Logical order vs physical execution

The eight phases define *meaning*, not the literal steps the executor runs. The planner (fig. 2) freely reorders work for speed, as long as the result is identical: it pushes WHERE predicates down into scans, uses an index to satisfy ORDER BY without a separate sort (fig. 16), and stops early for LIMIT. So “WHERE before SELECT” is a *semantic* guarantee; the physical plan in EXPLAIN may look nothing like the list.

7.5 PostgreSQL-specific niceties

- GROUP BY and ORDER BY may reference a SELECT alias or an output-column *number* (GROUP BY 1, ORDER BY 2) — a convenience SQL’s strict phase order would forbid. WHERE never can.
- DISTINCT ON (...) (section 6) is a PostgreSQL extension that rides on ORDER BY.
- WITH (CTEs) and FROM-subqueries act as phase-1 inputs: they produce named row sets the rest of the query consumes.

7.6 Reference: what each clause may use

Clause	Row columns	Aggregates	Window fns	SELECT aliases
WHERE	yes	no	no	no
GROUP BY	yes	no	no	PG: yes
HAVING	yes (grouped)	yes	no	no
SELECT	yes	yes	yes	— (defines them)
ORDER BY	yes	yes	yes	yes

Read this table top-to-bottom and it is just the phase order again: a clause may use a feature once the phase that produces it has run.

8 Aggregate functions

An aggregate folds many rows into one value (fig. 19). Aggregates run in the grouping phase of fig. 18 — which is why they are allowed in SELECT/HAVING but not in WHERE. Examples reuse emp (salaries: Alice 90000/dept 10, Bob 60000/dept 10, Carol 50000/dept 20, Dave 75000/dept NULL).

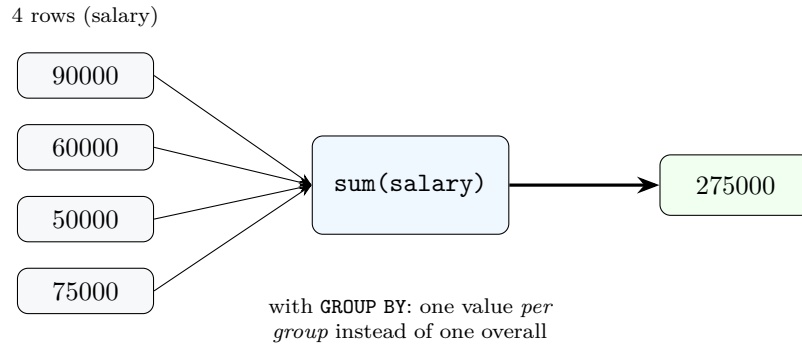


Figure 19: An aggregate collapses a set of rows into a single value (here $90000+60000+50000+75000 = 275000$). Run in the grouping phase of fig. 18: with no GROUP BY the whole table is one group and you get one row; with GROUP BY you get one value *per* group.

8.1 The common aggregates

Function	Result
count(*) / count(x) / count(DISTINCT x)	rows / non-NULL values / distinct values
sum, avg, min, max	total, mean, extremes (all skip NULL)
bool_and, bool_or (every)	all / any true
string_agg(x, sep)	values concatenated with a separator
array_agg(x) / jsonb_agg(x)	values collected into an array / JSON array
stddev, variance, corr, ...	descriptive statistics
percentile_cont/_disc ...	percentiles / median (ordered-set)
WITHIN GROUP	
mode() WITHIN GROUP	most frequent value (ordered-set)

```

1 SELECT count(*) AS n, sum(salary), avg(salary), min(salary), max(salary) FROM
   emp;
2 -- n | sum      | avg      | min      | max
3 -- 4 | 275000 | 68750 | 50000 | 90000

```

8.2 NULLs and the count() trio (flag)

Every aggregate except count(*) *ignores* NULL inputs. This is the source of endless confusion, so make it explicit:

```

1 SELECT count(*)           AS rows,           -- 4 (every row)
2        count(dept_id)     AS with_dept,      -- 3 (Dave's NULL is skipped)
3        count(DISTINCT dept_id) AS depts      -- 2 (10, 20; NULL never counts)
4        )
5 FROM emp;
6 SELECT avg(manager_id) FROM emp; -- averages the 3 non-NULL managers; Alice's
   NULL is NOT a zero

```

The practical trap: avg(x) divides by the count of *non-NULL* values, not by the row count. If you want NULL treated as 0, say so: avg(coalesce(x,0)).

8.3 GROUP BY: one value per group

```
1 SELECT dept_id, count(*) AS headcount, avg(salary) AS avg_sal
2 FROM emp GROUP BY dept_id;
3 -- dept_id / headcount / avg_sal
4 --      10 /          2 /    75000
5 --      20 /          1 /    50000
6 --      (NULL) /          1 /    75000      -- Dave's NULL dept is its own group
```

A column in SELECT must be either in GROUP BY or wrapped in an aggregate (the section 7 rule). Filter groups with HAVING:

```
1 SELECT dept_id, avg(salary) FROM emp GROUP BY dept_id HAVING avg(salary) >
   60000;
2 -- dept 10 (75000) and the NULL-dept group (75000); dept 20 (50000) is
   excluded
```

8.4 FILTER: conditional aggregates

Aggregate only the rows matching a predicate — cleaner than `sum(CASE WHEN ...)` and able to mix several conditions in one scan:

```
1 SELECT
2   count(*) AS total,      -- 4
3   count(*) FILTER (WHERE salary >= 60000) AS well_paid,  -- 3 (Alice, Bob,
   Dave)
4   avg(salary) FILTER (WHERE dept_id = 10) AS eng_avg      -- 75000
5 FROM emp;
```

8.5 Aggregates with an internal ORDER BY

`string_agg/array_agg` take an ORDER BY *inside* the call, controlling the order of the collected elements:

```
1 SELECT dept_id, string_agg(emp_name, ', ' ORDER BY salary DESC) AS names
2 FROM emp GROUP BY dept_id;
3 -- 10 -> 'Alice, Bob' 20 -> 'Carol' (NULL) -> 'Dave'
4 SELECT array_agg(emp_name ORDER BY emp_id) FROM emp;  -- {Alice,Bob,Carol,
   Dave}
```

8.6 Ordered-set aggregates: median, percentiles, mode

These need the data ordered, written with WITHIN GROUP (ORDER BY ...):

```
1 SELECT
2   percentile_cont(0.5) WITHIN GROUP (ORDER BY salary) AS median,      --
   67500
3   percentile_disc(0.9) WITHIN GROUP (ORDER BY salary) AS p90,      --
   90000
4   mode() WITHIN GROUP (ORDER BY dept_id) AS commonest_dept -- 10
5 FROM emp;
```

`percentile_cont` interpolates (median of {50,60,75,90}k = 67500); `percentile_disc` returns an actual present value.

8.7 Subtotals: GROUPING SETS, ROLLUP, CUBE

One query, several grouping levels. `ROLLUP` (a) adds a grand-total row; `CUBE` adds every combination; `GROUPING SETS` lists them explicitly.

```
1 SELECT dept_id, grouping(dept_id) AS is_total, count(*)
2 FROM emp GROUP BY ROLLUP (dept_id);
3 -- dept_id / is_total / count
4 --      10 /          0 /      2
5 --      20 /          0 /      1
6 --      (NULL) /          0 /      1      -- Dave's real NULL dept
7 --      (NULL) /          1 /      4      -- the grand-total row
```

Flag: the grand-total row also shows `dept_id = NULL`, colliding with Dave's genuine `NULL`. The `grouping()` function returns 1 for the synthetic total row, so use it to tell them apart.

8.8 Aggregate vs. window function (same function, two behaviours)

The defining difference: an aggregate *collapses* rows; the same function used `OVER (...)` *keeps* every row and attaches the group result (fig. 20).

```
1 SELECT dept_id, sum(salary)                                FROM emp GROUP BY dept_id
   ; -- 3 rows
2 SELECT emp_name, dept_id, sum(salary) OVER (PARTITION BY dept_id) FROM emp;
   -- 4 rows
```

aggregate + GROUP BY — collapses

sum(salary) ... GROUP BY dept_id

dept	sum
10	150000
20	50000
NULL	75000

3 rows — one per group

window function — keeps rows

sum(salary) OVER (PARTITION BY dept_id)

emp	dept	total
Alice	10	150000
Bob	10	150000
Carol	20	50000
Dave	NULL	75000

4 rows — all kept

Figure 20: The same function, two modes. As an *aggregate* under `GROUP BY` it collapses each group to one row. As a *window function* with `OVER (PARTITION BY ...)` it computes the same per-group total but *attaches* it to every original row, keeping them all. Use the first to summarize, the second to annotate.

8.9 A note on performance

With `GROUP BY`, the planner chooses between a `HashAggregate` (build a hash table keyed by the grouping columns — uses `work_mem`) and a `GroupAggregate` (requires sorted input, often from an index, and streams groups). `EXPLAIN` shows which; a hash aggregate that exceeds `work_mem` spills to disk, the same signal as fig. 15.

9 The write-ahead log (WAL)

The WAL is the mechanism behind three things PostgreSQL promises: durability, crash recovery, and replication. Its whole idea is one rule — *write the log before the data* — and almost everything else (checkpoints, LSNs, full-page writes, `synchronous_commit`) follows from making that rule cheap and correct.

9.1 The problem: durability without flushing everything

A committed transaction must survive a crash. The naive way — at every commit, `fsync` all the data pages it touched — is ruinously slow: those pages are scattered across the files, so it is random I/O, and a crash *during* those writes can leave a data file half-updated. WAL replaces that with a single *sequential* append.

9.2 The write-ahead rule (the invariant)

A modification first changes a page in `shared_buffers` (making it *dirty*) and appends a *WAL record* — a compact description of the change — to the WAL buffer. The invariant the system maintains (fig. 21):

A dirty data page may not be written to disk until the WAL record describing its change has been `fsync`'d first.

So at `COMMIT` only the WAL is forced to stable storage; the dirty data pages are written out lazily, later, by the checkpointer. Commit cost becomes one small sequential `fsync` instead of many random page writes.

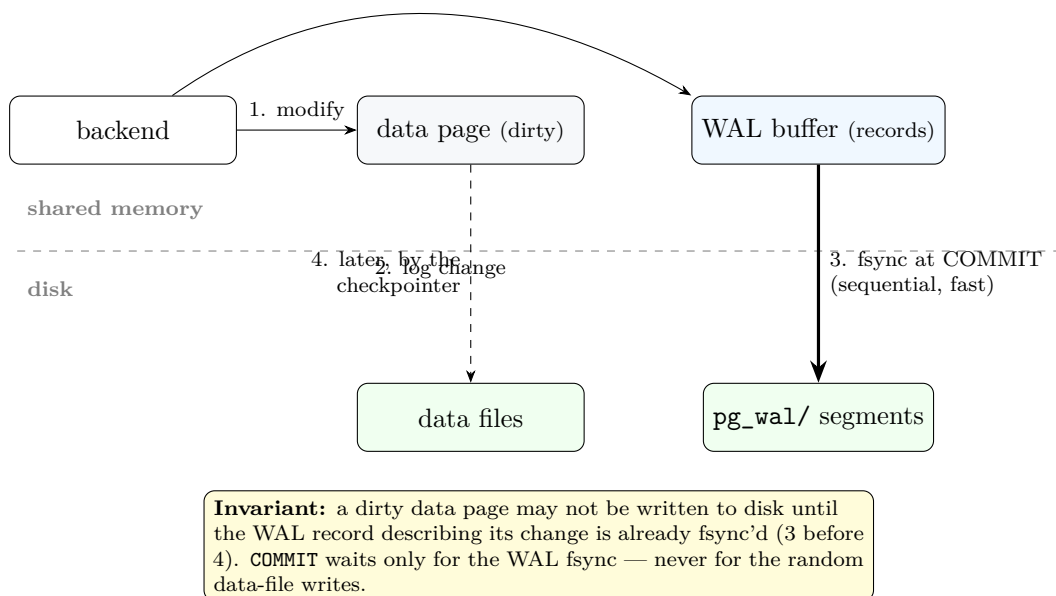


Figure 21: The write-ahead rule. A change first dirties a page in `shared_buffers` and appends a record to the WAL buffer. `COMMIT` flushes the WAL (a fast sequential `fsync` to `pg_wal/`) and returns; the modified data page is written to the data files much later by the checkpointer. Durability rides on the small sequential WAL write, not on flushing every touched page.

9.3 LSN: the WAL clock

Every byte of WAL has an address, the *LSN* (log sequence number), which only increases. Each data page header stores the LSN of the last WAL record that modified it (`pd_lsn`). That single number is how the invariant is enforced mechanically: before flushing a page with LSN ℓ , the system ensures the WAL has been fsync'd up to at least ℓ . LSNs also drive replication progress and recovery (below). You can read the current position:

```
1 SELECT pg_current_wal_lsn(); -- e.g. 3/AF12C8 (segment / offset)
```

9.4 The commit path: what fsync buys

At `COMMIT`, a WAL record marking the commit is written and the WAL is flushed up to its LSN. Once that fsync returns, the transaction is durable and the commit is acknowledged — *even though its data pages are still only in memory*. If the server crashes one instant later, recovery will reconstruct those pages from the WAL.

9.5 Checkpoints

A *checkpoint* writes all currently-dirty buffers out to the data files and records a checkpoint in the WAL. Its purpose is to bound two things: how much WAL must be kept, and how long recovery takes (recovery only ever replays from the last checkpoint). The checkpointer process runs them, triggered by `checkpoint_timeout` or by WAL volume reaching `max_wal_size`; `checkpoint_completion_target` spreads the writes out to avoid an I/O spike.

9.6 Crash recovery (REDO)

On restart after a crash, PostgreSQL enters recovery: it finds the last checkpoint and replays (*redoes*) every WAL record from there to the end, re-applying each change to its page (fig. 22). Replay is *idempotent* — a record whose change the page's stored LSN already reflects is skipped — so interrupted recovery can simply restart. When the WAL is exhausted, the data files are consistent and up to date, and the server opens.

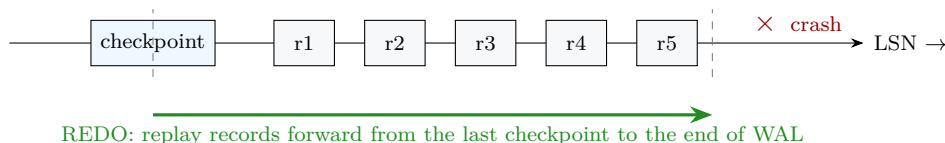


Figure 22: Crash recovery. WAL is one ever-growing stream addressed by *LSN* (log sequence number). On restart after a crash, PostgreSQL starts at the most recent *checkpoint* and *redoes* every following record in order, re-applying changes to the data pages until the WAL ends. Replay is idempotent: a record is skipped if the page's stored LSN already covers it, so recovering twice is harmless. WAL before the checkpoint is not needed for crash recovery and can be recycled or archived.

9.7 Full-page writes (torn-page protection)

An 8 kB page write is not atomic against an OS or power failure — a crash can leave a *torn page* (some 512-byte sectors new, some old). WAL records alone cannot fix a torn base page. So with

`full_page_writes = on` (the default), the *first* modification of a page after each checkpoint writes the *entire page image* into WAL; recovery restores that whole image before applying later delta records. This is why WAL volume spikes right after a checkpoint, and one reason very frequent checkpoints can cost more WAL, not less.

9.8 Durability and detail knobs

```
1 # postgresql.conf
2 wal_level = replica # minimal | replica | logical (how much WAL is written)
3 synchronous_commit = on # on: fsync WAL before ack; off: ack early, may lose recent
   commits
4 full_page_writes = on # torn-page protection
5 max_wal_size = 1GB # soft cap that triggers checkpoints
6 checkpoint_timeout = 5min
7 checkpoint_completion_target = 0.9
```

Two flags deserve emphasis:

- **`synchronous_commit = off`** trades durability for latency: a commit returns before its WAL is fsync'd, so a crash can lose the *most recent* committed transactions — but it never corrupts the database (the WAL still replays consistently). It is a safe *performance* knob, not a safety one.
- **`wal_level`** controls how much is logged: `minimal` permits optimizations like skipping WAL for a table created/truncated in the same transaction (recall section 5), but supports neither replication nor archiving; `replica` (default) enables physical replication and WAL archiving; `logical` additionally enables logical decoding.

9.9 What WAL also enables

Because the WAL is a complete, ordered description of every change, replaying it elsewhere reproduces the database. That single fact gives:

- **Physical / streaming replication** — ship WAL to standby servers that continuously replay it (a hot standby), the subject of the replication chapter.
- **Point-in-time recovery (PITR)** — a base backup plus archived WAL replayed up to a chosen LSN or timestamp.
- **Logical replication / CDC** — with `wal_level = logical`, WAL is decoded into row-level change events for selective replication or downstream consumers.

9.10 Inspecting WAL

```
1 SELECT pg_current_wal_lsn(); -- current write position
2 SELECT * FROM pg_stat_wal; -- WAL generation statistics
3 SELECT pg_walfile_name(pg_current_wal_lsn()); -- which 16 MB segment file
4 pg_waldump 000000010000000030000000A -- decode a segment, record by record (CLI)
```

Takeaway. WAL turns “make committed data durable” into “append a record to a sequential log and fsync that”; checkpoints bound the log and recovery time; LSNs keep data pages and the log in lockstep; full-page writes guard against torn pages; and the same log, replayed elsewhere, is the foundation of replication and PITR.

10 Triggers

A trigger runs a function automatically when a table changes. It is two pieces: a *trigger function* (returns the special type `trigger`) and a `CREATE TRIGGER` that binds the function to events on a table. Examples extend `emp` from section 2.

10.1 The taxonomy: timing $\times$ event $\times$ level

A trigger is defined along three axes:

Axis	Options	Notes
timing	BEFORE, AFTER, INSTEAD OF	INSTEAD OF is views only
event	INSERT, UPDATE [OF cols], DELETE, TRUNCATE	combine with OR
level	FOR EACH ROW, FOR EACH STATEMENT	TRUNCATE is statement-level only

10.2 Firing order

For one statement, triggers fire as in fig. 23: BEFORE STATEMENT, then per row BEFORE ROW $\rightarrow$ change $\rightarrow$ AFTER ROW, then AFTER STATEMENT. Multiple triggers on the same event fire *alphabetically by name* — so people prefix names (01_, 02_) to control order.

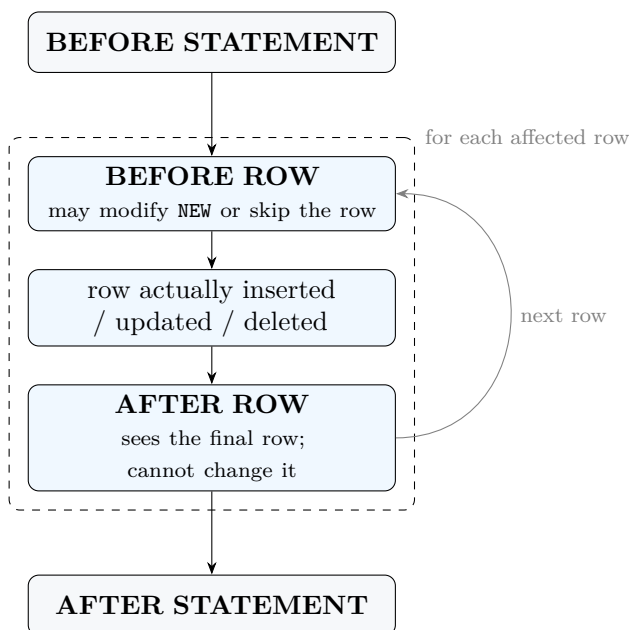


Figure 23: Trigger firing order for one statement. Statement-level triggers wrap the whole command; row-level triggers fire once *per affected row*, with BEFORE running before the change (where it can edit or veto the row) and AFTER running once the row is in place. When several triggers match the same event, they fire in *alphabetical order by trigger name*.

10.3 Inside the function: NEW, OLD, and TG_*

A trigger function takes no declared arguments; the server passes context in magic variables:

Variable	Meaning
NEW	the incoming row (INSERT, UPDATE); NULL on DELETE
OLD	the existing row (UPDATE, DELETE); NULL on INSERT
TG_OP	'INSERT' / 'UPDATE' / 'DELETE' / 'TRUNCATE'
TG_TABLE_NAME, TG_WHEN, TG_LEVEL	the table, BEFORE/AFTER, ROW/STATEMENT
TG_ARGV[], TG_NARGS	arguments passed in CREATE TRIGGER

10.4 Return-value semantics

The return value matters only for BEFORE ROW (and INSTEAD OF) triggers (fig. 24): return NEW to proceed (optionally modified), or NULL to skip the row. AFTER and statement-level triggers ignore it.

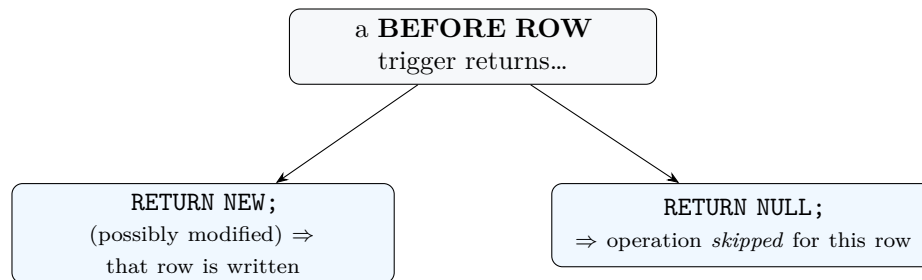


Figure 24: A BEFORE row trigger’s return value is control flow: return NEW (edited as you like) to let the write proceed, or return NULL to silently drop the operation for that row. For DELETE, return OLD to proceed. By contrast, AFTER row and statement-level triggers *ignore* the return value (return NULL by convention), and an INSTEAD OF trigger on a view returns a non-null row to report the action as done.

10.5 Worked examples

1. Auto-stamp updated_at (BEFORE UPDATE, modify the row).

```

1 ALTER TABLE emp ADD COLUMN updated_at timestampz DEFAULT now();
2
3 CREATE FUNCTION set_updated_at() RETURNS trigger AS $$
4 BEGIN
5     NEW.updated_at := now();    -- edit the row about to be written
6     RETURN NEW;
7 END;
8 $$ LANGUAGE plpgsql;
9
10 CREATE TRIGGER emp_set_updated_at
11 BEFORE UPDATE ON emp
12 FOR EACH ROW EXECUTE FUNCTION set_updated_at();

```

2. Audit log (AFTER, write to another table).

```

1 CREATE TABLE emp_audit (
2     audit_id  bigserial PRIMARY KEY,
3     op        text,
4     emp_id    int,
5     old_salary numeric,
6     new_salary numeric,

```

```

7      at          timestamptz DEFAULT now()
8  );
9
10 CREATE FUNCTION audit_emp() RETURNS trigger AS $$
11 BEGIN
12     IF TG_OP = 'INSERT' THEN
13         INSERT INTO emp_audit(op, emp_id, new_salary) VALUES ('INSERT', NEW.emp_id
14             , NEW.salary);
15     ELSIF TG_OP = 'UPDATE' THEN
16         INSERT INTO emp_audit(op, emp_id, old_salary, new_salary)
17             VALUES ('UPDATE', NEW.emp_id, OLD.salary, NEW.salary);
18     ELSIF TG_OP = 'DELETE' THEN
19         INSERT INTO emp_audit(op, emp_id, old_salary) VALUES ('DELETE', OLD.emp_id
20             , OLD.salary);
21     END IF;
22     RETURN NULL;    -- AFTER row: ignored
23 END;
24 $$ LANGUAGE plpgsql;
25
26 CREATE TRIGGER emp_audit_trg
27     AFTER INSERT OR UPDATE OR DELETE ON emp
28     FOR EACH ROW EXECUTE FUNCTION audit_emp();

```

3. Validate and normalize (BEFORE, can veto with an error).

```

1 CREATE FUNCTION validate_emp() RETURNS trigger AS $$
2 BEGIN
3     IF NEW.salary < 0 THEN
4         RAISE EXCEPTION 'salary cannot be negative (emp_id %)', NEW.emp_id;
5     END IF;
6     NEW.emp_name := btrim(NEW.emp_name);    -- normalize whitespace
7     RETURN NEW;
8 END;
9 $$ LANGUAGE plpgsql;
10
11 CREATE TRIGGER emp_validate
12     BEFORE INSERT OR UPDATE ON emp
13     FOR EACH ROW EXECUTE FUNCTION validate_emp();

```

10.6 WHEN: fire only when needed

A WHEN condition is checked before the function runs, so it avoids the call entirely. A classic use is auditing only *real* changes:

```

1 CREATE TRIGGER emp_audit_salary
2     AFTER UPDATE ON emp
3     FOR EACH ROW
4     WHEN (OLD.salary IS DISTINCT FROM NEW.salary)    -- skip no-op updates
5     EXECUTE FUNCTION audit_emp();

```

(IS DISTINCT FROM is the NULL-safe “not equal”.)

10.7 Statement-level triggers and transition tables

A row-level trigger that fires thousands of times is expensive. For bulk auditing, a `FOR EACH STATEMENT` trigger with *transition tables* sees all affected rows as two relations and handles them in one pass:

```
1 CREATE FUNCTION audit_emp_stmt() RETURNS trigger AS $$
2 BEGIN
3     INSERT INTO emp_audit(op, emp_id, old_salary, new_salary)
4     SELECT 'UPDATE', n.emp_id, o.salary, n.salary
5     FROM new_rows n JOIN old_rows o USING (emp_id)
6     WHERE o.salary IS DISTINCT FROM n.salary;
7     RETURN NULL;
8 END;
9 $$ LANGUAGE plpgsql;
10
11 CREATE TRIGGER emp_audit_stmt
12 AFTER UPDATE ON emp
13 REFERENCING OLD TABLE AS old_rows NEW TABLE AS new_rows
14 FOR EACH STATEMENT EXECUTE FUNCTION audit_emp_stmt();
```

10.8 Constraint triggers (deferrable)

`CREATE CONSTRAINT TRIGGER` is an `AFTER ROW` trigger that can be `DEFERRABLE INITIALLY DEFERRED`, firing at *transaction commit* rather than per statement — useful for cross-row or cross-table invariants that are only consistent once a whole transaction finishes.

10.9 Caveats (flags)

- **Per-row cost.** Row triggers fire for `COPY` and `INSERT` alike (section 5); a bulk load into a triggered table can be far slower. Prefer statement-level triggers, or disable triggers around big loads.
- **Hidden logic.** Triggers act at a distance — a plain `UPDATE` may cascade through several of them. Always check `\d emp` for what is attached.
- **Fires on no-op updates.** An `UPDATE` that changes nothing still fires `UPDATE` triggers unless a `WHEN` guard filters it.
- **Recursion.** A trigger that modifies its own table can re-fire itself; guard against infinite loops.

10.10 Managing triggers

```
1 \d emp -- lists triggers on the table
2 ALTER TABLE emp DISABLE TRIGGER emp_audit_trg; -- e.g. around a bulk load
3 ALTER TABLE emp ENABLE TRIGGER emp_audit_trg;
4 DROP TRIGGER emp_audit_trg ON emp;
```

11 DELETE vs TRUNCATE

Both remove rows, but they work so differently that the choice has real consequences for speed, space, locking, and triggers. Examples use `emp` from section 2.

11.1 What each does

- **DELETE** is row-level DML: it finds the rows matching **WHERE** (or all rows, with none) and marks each one dead under MVCC, firing row triggers and writing per-row WAL. Cost grows with the number of rows.
- **TRUNCATE** is a bulk operation: it empties an entire table by discarding its underlying file and allocating a fresh empty one. It does not look at rows at all, so it is essentially constant-time regardless of table size.

11.2 The key difference: storage

Figure 25 is the crux. **DELETE** leaves *dead tuples* that still occupy the file — bloat that only **VACUUM** reclaims — while **TRUNCATE** frees the space immediately.

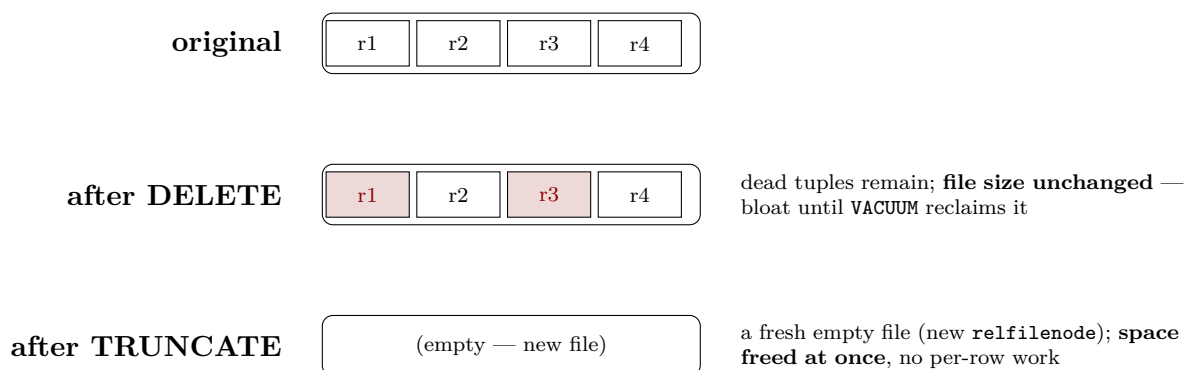


Figure 25: The core difference is what happens to storage. **DELETE** marks each matching row *dead* in place: the rows stop being visible, but the file does not shrink — that space is reclaimed only by **VACUUM** (and returned to the OS only by **VACUUM FULL**). **TRUNCATE** discards the table's file entirely and gives it a fresh empty one, so all the space is freed immediately and no row is touched individually.

A direct corollary: **DELETE FROM emp;** (no **WHERE**) empties the table but does it *row by row* — slow on a big table, and the file stays large. **TRUNCATE emp;** is the fast way to empty a whole table.

11.3 Side by side

Aspect	DELETE	TRUNCATE
Scope	any subset via WHERE	whole table(s) only
Speed	$O(\text{rows})$	$\approx$ constant, very fast
Dead tuples / bloat	yes — needs VACUUM	none — fresh file
Space to the OS	only via VACUUM FULL	freed immediately
Row triggers	fire per row	<i>not</i> fired
TRUNCATE triggers	—	statement-level, fired
RETURNING	supported	not supported
Reset identity	no	optional (RESTART IDENTITY)
Lock taken	ROW EXCLUSIVE (reads continue)	ACCESS EXCLUSIVE (blocks everything)
Referenced by an FK	per-row FK actions apply	needs CASCADE (or error)
Transactional (rollback)	yes	yes (in PostgreSQL)
WAL	per row	minimal

11.4 DELETE in practice

```
1 DELETE FROM emp WHERE dept_id = 20;           -- removes Carol;
   fires triggers
2 DELETE FROM emp WHERE dept_id = 20 RETURNING emp_id, emp_name; -- get the
   removed rows back
3 DELETE FROM emp;                               -- all rows,
   still one-by-one (bloat)
```

11.5 TRUNCATE specifics

```
1 TRUNCATE emp;                                   -- empty the table, free its space
2 TRUNCATE emp RESTART IDENTITY;                 -- also reset its owned sequences (e.g.
   an id serial)
3 TRUNCATE emp, emp_audit;                       -- several tables atomically
4 TRUNCATE dept CASCADE;                         -- also truncate tables that reference
   dept (emp.dept_id -> dept)
```

Flags:

- **Foreign keys.** TRUNCATE dept *errors* because emp references it; you must add CASCADE (which truncates emp too) or truncate both explicitly. DELETE, by contrast, follows each row's FK action (ON DELETE CASCADE/RESTRICT).
- **Locking.** TRUNCATE takes an ACCESS EXCLUSIVE lock — it blocks even SELECTs on the table until it commits, so on a hot table it is a brief outage. DELETE takes only ROW EXCLUSIVE, letting reads and most writes continue.
- **No WHERE, no RETURNING.** TRUNCATE cannot filter or report rows; it is all-or-nothing.

11.6 Both are transactional in PostgreSQL (flag)

Unlike some databases (where TRUNCATE is an implicit-commit DDL operation), PostgreSQL's TRUNCATE is fully transactional: it can be wrapped in a transaction and ROLLBACK'd, and it is crash-safe.

```
1 BEGIN;
2 TRUNCATE emp;
3 -- ... changed your mind ...
4 ROLLBACK;           -- emp is back, untouched
```

11.7 When to use which

- Removing a *subset* of rows — only DELETE can.
- Need row triggers, RETURNING, or to keep readers unblocked — DELETE.
- Emptying a *whole* table fast, reclaiming its space, and/or resetting identity — TRUNCATE.
- Either way inside a transaction is safe to roll back.

12 Foreign keys

A foreign key (FK) enforces *referential integrity*: a value in a child table must match an existing key in a parent table (or be NULL). Examples use emp/dept from section 2, where emp.dept_id references dept and emp.manager_id references emp itself.

12.1 Declaring a foreign key

```
1 -- inline (column constraint); REFERENCES dept alone targets dept's PRIMARY
  KEY
2 CREATE TABLE emp (
3   emp_id      int PRIMARY KEY,
4   emp_name    text NOT NULL,
5   dept_id     int REFERENCES dept (dept_id) ON DELETE SET NULL,
6   manager_id  int REFERENCES emp (emp_id) ON DELETE SET NULL
7 );
8
9 -- as a named table constraint (add later, or for composite keys)
10 ALTER TABLE emp
11   ADD CONSTRAINT emp_dept_fk FOREIGN KEY (dept_id)
12   REFERENCES dept (dept_id) ON DELETE CASCADE;
```

The referenced column(s) *must* carry a PRIMARY KEY or UNIQUE constraint (fig. 26). Composite keys reference a matching set of columns: FOREIGN KEY (a, b) REFERENCES parent (a, b).

12.2 The integrity rule and NULLs

A NULL foreign-key value is always allowed — it means “no parent” and is not checked. For composite keys the default MATCH SIMPLE accepts a row if *any* FK column is NULL; MATCH FULL requires all FK columns to be NULL together or all non-NULL.

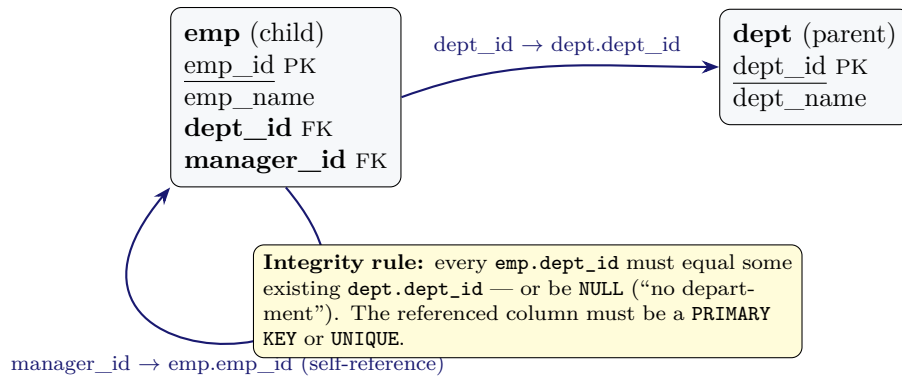


Figure 26: A foreign key makes a *child* column point at a *parent*’s unique key. `emp.dept_id` references `dept.dept_id`, and `emp.manager_id` references `emp.emp_id` (a self-reference). PostgreSQL then forbids any child value that has no matching parent row.

12.3 Referential actions: ON DELETE / ON UPDATE

These say what happens to children when a parent key is deleted or changed (fig. 27):

Action	Effect on child rows
NO ACTION (default)	error if children exist; the check can be deferred to end of statement/transaction
RESTRICT	error if children exist; checked <i>immediately</i> (cannot be deferred)
CASCADE	delete (or update) the child rows too
SET NULL	set the child FK column(s) to NULL
SET DEFAULT	set the child FK column(s) to their DEFAULT

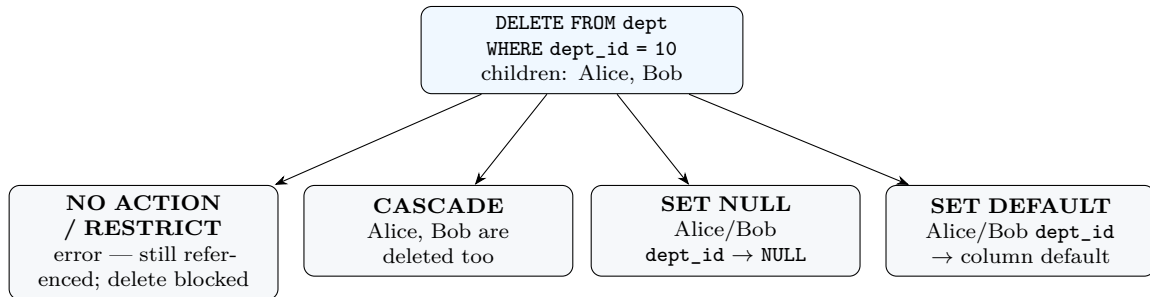


Figure 27: The `ON DELETE` action decides what happens to child rows when their parent is deleted (`ON UPDATE` works the same way for changes to the parent key). `NO ACTION/RESTRICT` block the delete; `CASCADE` deletes the children; `SET NULL` / `SET DEFAULT` rewrite the child’s foreign-key column.

12.4 NO ACTION vs RESTRICT (the subtle one)

Both reject a delete that would orphan children, but they differ in *timing*. `RESTRICT` fires the moment the row is deleted. `NO ACTION` defers the check to the end of the statement (and can be deferred to commit), which lets a later statement in the same command “fix” things first — e.g. a cascade elsewhere that removes the children. Default to `NO ACTION` unless you specifically want the earlier check.

12.5 How a foreign key is enforced

An FK is not magic: PostgreSQL implements it with system-level *constraint triggers* (section 10). Inserting or updating a child fires an **AFTER** trigger that checks the parent exists; deleting or updating a parent fires triggers on the parent that apply the action to children. This is why FK checks are per-row and why they participate in transactions like any other trigger.

12.6 Index your foreign-key columns (gotcha)

The *parent* side is always indexed (it must be **PRIMARY KEY**/**UNIQUE**). The *child* side is **not** indexed automatically. Without an index on the child FK column, every parent **DELETE**/key-**UPDATE** must sequentially scan the child to find references — a notorious source of slow deletes and lock contention. Add it yourself:

```
1 CREATE INDEX ON emp (dept_id);      -- index the CHILD fk column; PostgreSQL
   won't do this for you
```

12.7 Deferrable constraints

A **DEFERRABLE** FK can postpone its check to transaction commit, which is what makes circular references and bulk reordering possible:

```
1 ALTER TABLE emp ADD CONSTRAINT emp_dept_fk FOREIGN KEY (dept_id)
2   REFERENCES dept (dept_id) DEFERRABLE INITIALLY DEFERRED;
3
4 BEGIN;
5   -- temporarily inconsistent rows are fine until COMMIT
6   INSERT INTO emp ...;      -- child before its parent, say
7   INSERT INTO dept ...;
8 COMMIT;                     -- all FK checks run now
9 -- or per-transaction: SET CONSTRAINTS ALL DEFERRED;
```

12.8 Adding a foreign key to a big table

Adding an FK normally scans and locks the whole child table to validate existing rows. For a large, live table, split it: add it **NOT VALID** (cheap — only new rows are checked), then **VALIDATE** separately under a weaker lock.

```
1 ALTER TABLE emp ADD CONSTRAINT emp_dept_fk FOREIGN KEY (dept_id)
2   REFERENCES dept (dept_id) NOT VALID;      -- fast; future writes are
   enforced
3 ALTER TABLE emp VALIDATE CONSTRAINT emp_dept_fk; -- scans existing rows later,
   without an exclusive lock
```

12.9 Managing

```
1 \d emp -- shows FK constraints and "Referenced by"
2 ALTER TABLE emp DROP CONSTRAINT emp_dept_fk;
```

Takeaway. FKs guarantee that references resolve; pick the **ON DELETE**/ **ON UPDATE** action that matches your data's life-cycle; always index the child column; and reach for **DEFERRABLE** / **NOT VALID** when transactions or migrations need the check relaxed in time.

13 Transactions

A transaction is a unit of work that is all-or-nothing and isolated from others. This chapter goes from the basics through MVCC, isolation levels, and locking — the machinery that lets many sessions hit the same data at once and still get correct answers. Examples use `emp` from section 2.

13.1 BEGIN, COMMIT, ROLLBACK, autocommit

By default `psql`/drivers run in *autocommit*: each statement is its own transaction. To group statements, wrap them:

```
1 BEGIN;
2 UPDATE emp SET salary = salary * 1.10 WHERE dept_id = 10;
3 UPDATE emp SET salary = salary * 1.05 WHERE dept_id = 20;
4 COMMIT;      -- both take effect, or neither (use ROLLBACK to discard)
```

If anything fails (or you `ROLLBACK`), none of the changes are visible to anyone — that is *atomicity*.

13.2 ACID, and how PostgreSQL provides each

Property	In PostgreSQL
Atomicity	abort/rollback discards all of a transaction's changes
Consistency	constraints (FKs section 12, checks) plus your logic hold invariants
Isolation	MVCC snapshots give each transaction a consistent view (below)
Durability	committed changes survive crashes via the WAL (section 9)

13.3 Savepoints: partial rollback

A `SAVEPOINT` marks a point you can roll back *to* without aborting the whole transaction:

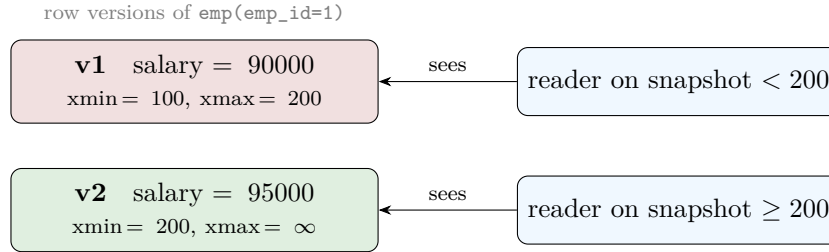
```
1 BEGIN;
2 INSERT INTO emp (emp_id, emp_name, salary) VALUES (9, 'Ivan', 50000);
3 SAVEPOINT s1;
4 UPDATE emp SET salary = -1 WHERE emp_id = 9;      -- mistake
5 ROLLBACK TO SAVEPOINT s1;                        -- undo just the bad UPDATE
6 COMMIT;                                           -- Ivan's INSERT survives
```

13.4 MVCC: snapshots and row versions

PostgreSQL gives concurrency through *multiversion concurrency control*: an `UPDATE` or `DELETE` does not overwrite a row, it writes a new *version* and marks the old one (fig. 28). Every version carries `xmin/xmax` (the transactions that created/retired it), and each transaction's *snapshot* decides which versions it can see. The payoff: **readers and writers do not block each other**. The cost: superseded versions are *dead tuples* that `VACUUM` must later reclaim (section 11).

13.5 Isolation levels

Isolation controls which concurrency *anomalies* you can observe. The standard names four; PostgreSQL's levels and what they prevent:



An UPDATE by transaction 200 does not overwrite v1 — it writes a *new* version v2 and stamps v1's xmax. Each reader sees the version valid in *its* snapshot, so **writers never block readers and readers never block writers**. Once no live snapshot needs v1, it becomes a dead tuple for VACUUM (section 11).

Figure 28: Multiversion concurrency control (MVCC). Every row version carries xmin (the transaction that created it) and xmax (the transaction that deleted/superseded it). A transaction's *snapshot* decides which versions are visible, which is how PostgreSQL gives each transaction a consistent view without locking out others.

Level	Dirty read	Non-repeatable	Phantom	Serialization anomaly
READ COMMITTED (default)	never	possible	possible	possible
REPEATABLE READ	never	no	no	possible
SERIALIZABLE	never	no	no	no

Two PostgreSQL specifics: it *never* allows dirty reads (READ UNCOMMITTED is treated as READ COMMITTED), and its REPEATABLE READ is true snapshot isolation, so it also blocks phantoms — stronger than the SQL standard's minimum.

13.6 READ COMMITTED vs REPEATABLE READ

The practical difference is *when the snapshot is taken* (fig. 29): READ COMMITTED takes a new snapshot at the *start of each statement*, so it sees other transactions' commits as they land; REPEATABLE READ takes *one* snapshot at transaction start and keeps it.

```

1 BEGIN ISOLATION LEVEL REPEATABLE READ;
2 SELECT salary FROM emp WHERE emp_id = 1;    -- 90000
3 -- (another session updates and commits salary = 95000)
4 SELECT salary FROM emp WHERE emp_id = 1;    -- still 90000 in this transaction
5 COMMIT;
  
```

13.7 SERIALIZABLE and serialization failures

SERIALIZABLE uses Serializable Snapshot Isolation (SSI): it tracks read/write dependencies between concurrent transactions and, if they could produce a result no serial order would, aborts one with SQLSTATE 40001 (serialization_failure). The contract shifts to the application: **be ready to retry**.

```

1 -- application retry loop (pseudo-code)
2 loop:
3   BEGIN ISOLATION LEVEL SERIALIZABLE;
  
```

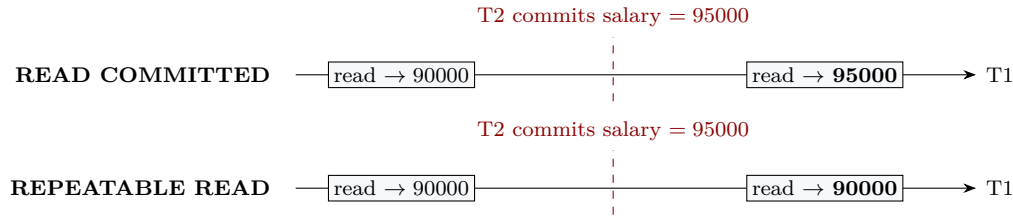


Figure 29: The same interleaving under two isolation levels: T1 reads a salary, T2 commits a change, T1 reads again. Under `READ COMMITTED` each statement takes a fresh snapshot, so the second read sees the new value (a *non-repeatable read*). Under `REPEATABLE READ` the whole transaction uses one snapshot taken at its start, so T1 keeps seeing the original value — reads are repeatable.

```

4  ... read and write ...
5  COMMIT;                -- on SQLSTATE 40001: ROLLBACK and retry the
    whole block

```

13.8 Row locking and lost updates

The classic lost-update bug: two sessions read a value, both compute a new one, both write — one update is lost. Under `READ COMMITTED`, prevent it by locking the row while you work, or by updating atomically:

```

1  BEGIN;
2  SELECT salary FROM emp WHERE emp_id = 1 FOR UPDATE;  -- lock this row; others
    wait
3  UPDATE emp SET salary = salary + 1000 WHERE emp_id = 1;
4  COMMIT;
5
6  -- or skip the read entirely and let the database do it atomically:
7  UPDATE emp SET salary = salary + 1000 WHERE emp_id = 1;

```

Lock strengths range from `FOR KEY SHARE` (weakest) through `FOR SHARE`, `FOR NO KEY UPDATE`, to `FOR UPDATE` (strongest). When two transactions wait on each other's locks, PostgreSQL detects the *deadlock* and aborts one with an error.

13.9 Long transactions and idle-in-transaction (gotcha)

A transaction holds its snapshot until it ends. A transaction left open — especially *idle in transaction* — pins old row versions so `VACUUM` cannot reclaim them, causing bloat and, eventually, transaction-id wraparound pressure. Keep transactions short, and cap stragglers:

```

1  SET idle_in_transaction_session_timeout = '30s'; -- auto-abort idle open transactions

```

13.10 DDL is transactional (flag)

Unlike many databases, PostgreSQL runs *DDL* inside transactions too — you can add a column and roll it back:

```

1  BEGIN;
2  ALTER TABLE emp ADD COLUMN tmp int;
3  ROLLBACK;  -- the column never existed; great for safe migrations

```


A few commands cannot run inside a transaction block — `VACUUM`, `CREATE INDEX CONCURRENTLY`, `CREATE DATABASE` — because they manage their own transactional behavior.

14 MVCC in depth

Section 13 introduced MVCC at the level of snapshots; this chapter goes down to the tuple. The payoff is understanding *bloat*, *HOT* updates, and why `VACUUM` — including freezing — is mandatory rather than optional.

14.1 The tuple header

Each row version (a *tuple*) stores, besides its column data, a header with:

Field	Meaning
<code>xmin</code>	transaction that <i>created</i> this version
<code>xmax</code>	transaction that <i>deleted/superseded</i> it (0/∞ if still live)
<code>cmin/cmax</code>	command ids, for visibility <i>within</i> one transaction
<code>ctid</code>	location of the next version (the version-chain pointer)
<code>infomask</code> (hint bits)	cached commit/abort status of <code>xmin/xmax</code>

14.2 Visibility

A tuple is visible to a snapshot, roughly, when its `xmin` is a *committed* transaction that the snapshot can see, *and* its `xmax` is unset, aborted, or a transaction the snapshot cannot see. Commit status comes from the commit log (`pg_xact`); the first reader to confirm it caches the answer in the tuple's *hint bits* so later reads are cheap.

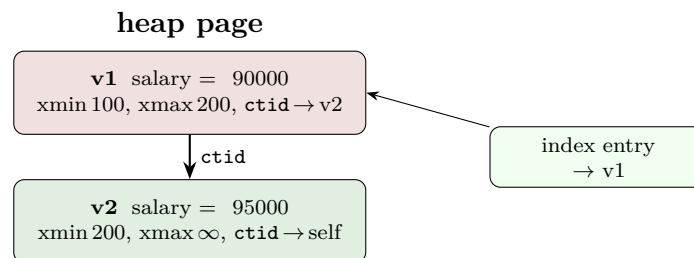
14.3 Version chains and HOT updates

An `UPDATE` writes a new version and links the old one to it through `ctid` (fig. 30). Normally each new version also needs new index entries. The *HOT* (heap-only tuple) optimization avoids that: if the update changes no indexed column and the new version fits on the same page, no index entry is added and the new version is found by following the chain — a large win against index bloat.

14.4 Dead tuples, bloat, and VACUUM

A version no live snapshot can ever see again is a *dead tuple*. Dead tuples still occupy pages — that is *bloat* (section 11). `VACUUM` is the cleanup:

- **`VACUUM`** marks dead tuples' space reusable (recorded in the *free space map*) and removes dead index entries. It does *not* shrink the file; the space is reused by future inserts/updates. It runs online.
- **`VACUUM FULL`** rewrites the whole table compactly and returns space to the OS, but takes an `ACCESS EXCLUSIVE` lock — a maintenance-window tool.
- **`autovacuum`** runs `VACUUM/ANALYZE` automatically based on how many rows changed; it is the normal mechanism and should be left on.



HOT (heap-only tuple): if the `UPDATE` changes no indexed column and v2 fits on the *same page*, no new index entry is made — the index still points at v1 and the scan follows the `ctid` chain to v2. This is how PostgreSQL avoids index bloat on hot-updated rows.

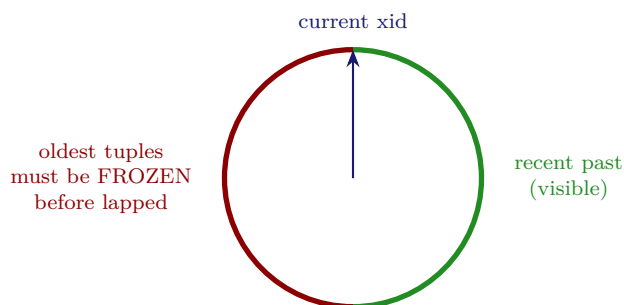
Figure 30: Row versions form a chain through the `ctid` pointer in each tuple header (which also holds `xmin/xmax`). An `UPDATE` appends v2 and links v1 to it. With a *HOT* update the index is left untouched and v2 is reached by following the chain; otherwise the index gets a new entry pointing directly at v2.

14.5 The visibility map

Alongside each table, a *visibility map* marks pages where every tuple is visible to all transactions. It lets `VACUUM` skip clean pages and lets *index-only scans* return data straight from the index without visiting the heap — a major read optimization.

14.6 Transaction-id wraparound and freezing

Transaction ids are 32-bit and *wrap around* (fig. 31). To stop an old row from one day looking like it is from the future (and disappearing), `VACUUM` *freezes* sufficiently old tuples — effectively stamping them “always committed, always visible” — and advances the table’s `relfrozenxid`. Autovacuum escalates to a forced freezing vacuum as `age(relfrozenxid)` approaches `autovacuum_freeze_max_age`; ignoring it eventually forces the database read-only to protect itself. This is the deep reason `VACUUM` is mandatory.



Transaction ids are 32-bit and *wrap* at 2^{32} ; only the ~ 2 billion ids “behind” the current one are seen as past. Autovacuum **freezes** sufficiently old tuples (advancing `relfrozenxid`) so they remain visible forever; if freezing falls too far behind, PostgreSQL forces it to prevent wraparound.

Figure 31: Transaction-id wraparound. Visibility is defined modulo 2^{32} , so a tuple too old and *unfrozen* could suddenly appear to be from the future and vanish. Freezing (by autovacuum) is what keeps old rows permanently visible — the reason `VACUUM` is not optional.

14.7 Seeing it

```
1 SELECT xmin, xmax, ctid, * FROM emp WHERE emp_id = 1; -- system columns are queryable
2 SELECT relname, n_dead_tup, n_live_tup, last_autovacuum
3 FROM pg_stat_user_tables WHERE relname = 'emp'; -- dead-tuple / autovacuum stats
4 SELECT age(relfrozenxid) FROM pg_class WHERE relname = 'emp'; -- how close to forced
   freeze
```

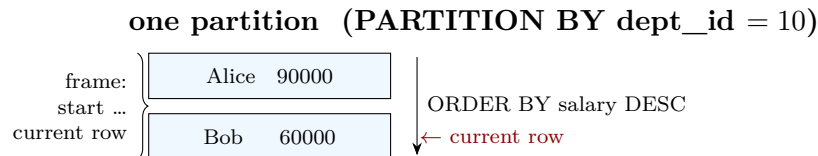
Takeaway. MVCC buys lock-free reads by keeping multiple versions; the bill is dead tuples and the duty to `VACUUM` — to reclaim space, keep indexes lean (helped by HOT), maintain the visibility map, and freeze old rows before the xid clock laps them.

15 Window functions

A window function computes a value across a set of rows *related to the current row* — but, unlike an aggregate, it keeps every row (fig. 20). It is the tool for rankings, running totals, moving averages, and “compare each row to its group”. Examples use `emp` from section 2.

15.1 Anatomy: OVER (PARTITION BY ... ORDER BY ... frame)

The `OVER` clause has three parts (fig. 32): `PARTITION BY` splits rows into independent groups, `ORDER BY` orders rows within a group, and the *frame* chooses which of those rows feed the function for each row. Window functions are evaluated in the `SELECT` phase (section 7), so they cannot appear in `WHERE/HAVING`.



The window function is evaluated over the *frame* (here, partition start to the current row — a running total). `PARTITION BY` picks the band, `ORDER BY` orders rows within it, and the *frame* selects which of those rows feed the function for each current row. Other partitions (dept 20, the `NULL` dept) are computed independently.

Figure 32: The three parts of a window. Unlike a `GROUP BY` aggregate (fig. 20), a window function keeps every row and computes its value from a *frame* of neighbouring rows defined by `PARTITION BY`, `ORDER BY`, and the frame clause.

15.2 Ranking functions

```
1 SELECT emp_name, salary,
2        row_number() OVER w AS rn,
3        rank()      OVER w AS rnk,
4        dense_rank() OVER w AS drnk
5 FROM emp
6 WINDOW w AS (ORDER BY salary DESC); -- a named window, reused above
7 -- emp_name | salary | rn | rnk | drnk
8 -- Alice   | 90000 | 1 | 1 | 1
```

```

9  -- Dave      / 75000 / 2 / 2 / 2
10 -- Bob       / 60000 / 3 / 3 / 3
11 -- Carol     / 50000 / 4 / 4 / 4

```

On ties they differ: `row_number` is always distinct (arbitrary among equals), `rank` leaves gaps after a tie (1,1,3), `dense_rank` does not (1,1,2). Also useful: `ntile(n)`, `percent_rank()`, `cume_dist()`.

15.3 Offset and positional functions

```

1 SELECT emp_name, salary,
2        lag(salary) OVER (ORDER BY salary DESC) AS next_higher,
3        lead(salary) OVER (ORDER BY salary DESC) AS next_lower,
4        first_value(emp_name) OVER (ORDER BY salary DESC) AS top_earner
5 FROM emp;

```

`lag/lead` reach to neighbouring rows; `first_value/last_value/nth_value` pull from frame positions.

15.4 Aggregates as windows: running totals, shares

Any aggregate works as a window function:

```

1 SELECT emp_name, dept_id, salary,
2        sum(salary) OVER (PARTITION BY dept_id) AS
3        dept_total,
4        round(100.0 * salary / sum(salary) OVER (), 1) AS
5        pct_of_company,
6        sum(salary) OVER (ORDER BY salary DESC
7                          ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS
8        running_total
9 FROM emp;

```

15.5 Frames: ROWS vs RANGE (a gotcha)

With an `ORDER BY` but no explicit frame, the *default* frame is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` — which includes *all peers* tied with the current row. For a strict row-by-row running total use `ROWS`:

```

1 ... sum(x) OVER (ORDER BY k) -- RANGE
   default: ties lumped together
2 ... sum(x) OVER (ORDER BY k ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
   -- true running total
3 ... avg(x) OVER (ORDER BY k ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
   -- 3-row moving average

```

15.6 Top-*N* per group

Window ranking plus an outer filter is the idiomatic “top row per group” (the window itself can’t go in `WHERE`, so wrap it):

```

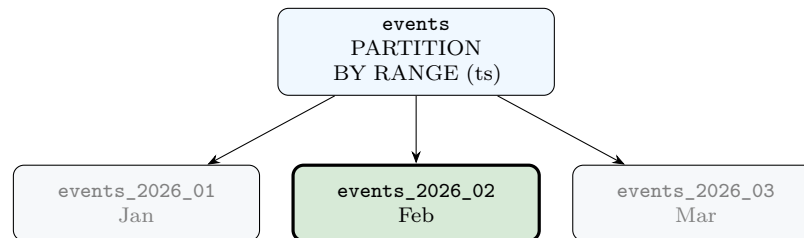
1 SELECT * FROM (
2   SELECT emp_name, dept_id, salary,
3     row_number() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rn
4   FROM emp
5 ) ranked
6 WHERE rn = 1;      -- highest-paid per department (cf. DISTINCT ON, sec. on
                    ordering)

```

Takeaway. Reach for a window function whenever you need a per-group or neighbour-relative value *without* collapsing rows; mind the default **RANGE** frame, and remember windows run after grouping, so filter their results in an outer query.

16 Table partitioning

Partitioning splits one logical table into many physical *partitions* by a key, so the database (and you) can work with manageable pieces. PostgreSQL’s *declarative* partitioning (version 10+) makes the parent a router and each child a real table (fig. 33).



A query `WHERE ts >= '2026-02-10'` is *pruned* to the Feb partition (green); the planner never touches January or March. The parent `events` holds no data itself — it routes rows to the child that matches the partition key.

Figure 33: Declarative partitioning. One logical table `events` is physically split into child partitions by a key (here a monthly **RANGE** on `ts`). Inserts are routed to the right child; queries with a key predicate are *pruned* to just the relevant partitions, and a whole partition can be dropped or detached instantly.

16.1 Why partition

- **Pruning** — queries that filter on the partition key scan only the relevant partitions, not the whole table.
- **Instant bulk removal** — dropping or detaching a partition deletes a whole slice (e.g. last year’s data) with no row-by-row **DELETE** or bloat.
- **Smaller pieces** — per-partition indexes and **VACUUM** stay cheaper; bulk loads can target one partition.

16.2 Declarative partitioning: RANGE, LIST, HASH

```

1 CREATE TABLE events (
2     id          bigint,
3     ts          timestamptz NOT NULL,
4     payload     jsonb
5 ) PARTITION BY RANGE (ts);
6
7 CREATE TABLE events_2026_01 PARTITION OF events
8     FOR VALUES FROM ('2026-01-01') TO ('2026-02-01');    -- upper bound is
9     EXCLUSIVE
10
11 CREATE TABLE events_2026_02 PARTITION OF events
12     FOR VALUES FROM ('2026-02-01') TO ('2026-03-01');
13
14 CREATE TABLE events_default PARTITION OF events DEFAULT;  -- catches anything
15     unmatched
16
17 -- LIST: by discrete values
18 CREATE TABLE people (dept_id int, ...) PARTITION BY LIST (dept_id);
19 CREATE TABLE people_eng PARTITION OF people FOR VALUES IN (10);
20
21 -- HASH: spread evenly across N partitions
22 CREATE TABLE t (...) PARTITION BY HASH (id);
23 CREATE TABLE t_p0 PARTITION OF t FOR VALUES WITH (MODULUS 4, REMAINDER 0);

```

16.3 Partition pruning

The planner drops partitions a query cannot match:

```

1 EXPLAIN SELECT * FROM events WHERE ts >= '2026-02-10';
2 -- Seq Scan on events_2026_02 events -- only this child; Jan/Mar pruned away

```

Pruning happens at plan time for constant predicates, and at execution time for parameters.

16.4 Attaching and detaching

The operational superpower: load or archive a partition as a unit.

```

1 -- build next month offline, then make it live atomically
2 CREATE TABLE events_2026_04 (LIKE events INCLUDING DEFAULTS);
3 -- ... bulk COPY into events_2026_04 ...
4 ALTER TABLE events ATTACH PARTITION events_2026_04
5     FOR VALUES FROM ('2026-04-01') TO ('2026-05-01');
6
7 -- retire old data instantly (no DELETE, no bloat)
8 ALTER TABLE events DETACH PARTITION events_2026_01;
9 DROP TABLE events_2026_01;

```

16.5 Indexes and constraints

An index created on the parent is created on every partition (current and future). A **unique constraint or primary key must include the partition key** — PostgreSQL cannot enforce global uniqueness across partitions otherwise:

```

1 CREATE TABLE events (... , PRIMARY KEY (id, ts)) PARTITION BY RANGE (ts);  --
   key includes ts
2 CREATE INDEX ON events (ts);  -- propagated to all partitions

```

16.6 Gotchas (flags)

- **No global unique on a non-key column.** You cannot make `id` alone unique across partitions; the partition key must be part of every unique constraint.
- **Row migration.** An `UPDATE` that changes the partition key moves the row to another partition (allowed, but it is a delete + insert under the hood).
- **Too many partitions.** Thousands of partitions add planning overhead; keep the count reasonable and let pruning do the work.
- **Default partition** simplifies inserts but blocks adding a new partition whose range overlaps rows already sitting in the default.

Takeaway. Partition large, time- or key-sliced tables to get pruning and instant slice management; put the partition key in your primary/unique keys; and prefer `DETACH/DROP` of whole partitions over giant `DELETES`.

17 The type system

PostgreSQL is strongly and statically typed: every value, column, and expression has a type, and that type decides which operators and functions apply. The built-in set is unusually rich (fig. 34), and you can add your own (section 18).

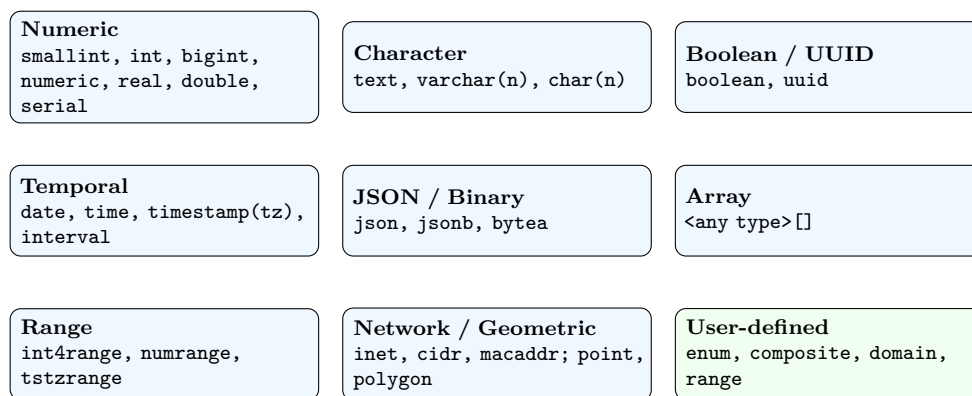


Figure 34: PostgreSQL’s type system is broad: exact and floating numerics, several character and temporal types, first-class `jsonb`, arrays of *any* type, ranges, network and geometric types, and a clean path to your own (section 18). Every value carries a type, and operators/functions are resolved by it.

17.1 Numeric: exact vs floating (flag)

Type	Use
smallint/int/bigint	exact integers (2/4/8 bytes)
numeric(p,s)	exact decimal — money, anything that must not round
real, double precision	binary floating point — fast, but inexact
serial, bigserial, GENERATED ...AS IDENTITY	auto-incrementing keys (sequence-backed)

Flag: never store money in `real/double` — `0.1+0.2 != 0.3` in binary float. Use `numeric`.

17.2 Character: prefer text

`text`, `varchar(n)`, and `char(n)` are the same machinery; `text` and `varchar` perform identically. Use `text` (optionally with a `CHECK` on length) unless a hard limit is a real requirement. Avoid `char(n)` — it pads with spaces.

17.3 Temporal: always timestamptz

`date`, `time`, `timestamp`, `timestamptz`, `interval`. **Flag:** prefer `timestamptz`; it stores an absolute instant (UTC) and converts to the session time zone on display. `timestamp` (without time zone) silently drops the zone and is a frequent bug source.

17.4 JSON and JSONB

`json` keeps the exact text; `jsonb` parses to a binary form that is deduplicated, indexable, and supports containment. Prefer `jsonb` for storage/query.

```
1 SELECT '{"a":1,"b":[2,3]}':::jsonb -> 'b' -> 0;      -- navigate: 2 (-> keeps
   json, ->> gives text)
2 SELECT data FROM t WHERE data @> '{"active":true}'; -- containment, GIN-
   indexable
3 CREATE INDEX ON t USING gin (data);                  -- index jsonb for @>, ?,
   path queries
```

17.5 Arrays

Any type can be an array. **Flag:** array indexing is *1-based*.

```
1 SELECT ARRAY[10,20,30] AS a, (ARRAY[10,20,30])[1] AS first; -- first = 10
2 SELECT * FROM emp WHERE dept_id = ANY(ARRAY[10,20]);        -- IN-list as an
   array
3 SELECT array_agg(emp_name ORDER BY emp_id) FROM emp;        -- build one (
   cf. aggregates)
4 SELECT unnest(ARRAY['a','b']) AS x;                          -- expand back
   into rows
```


17.6 Ranges

A range is a pair of bounds of some subtype (`int4range`, `numrange`, `tstzrange`), with operators like containment `@>` and overlap `&&`, and `EXCLUDE` constraints to forbid overlapping bookings.

```
1 SELECT int4range(1,10) @> 5, tstzrange(now(), now()+interval '1h') &&
   tstzrange(now(), 'infinity');
```

17.7 Casting

Convert with `CAST(x AS t)` or the shorthand `x::t`. Casts come in three strengths: *implicit* (applied automatically, kept deliberately rare to avoid ambiguity), *assignment* (on `INSERT/UPDATE`), and *explicit* (you ask). When in doubt, cast explicitly:

```
1 SELECT '2026-06-23'::date, 42::text, '3.14'::numeric, '{1,2}'::int[];
```

17.8 Under the hood

Every type — built-in or yours — is a row in `pg_type` with input/output functions that parse and render its text form. That uniformity is what lets user-defined types (section 18) behave exactly like built-ins.

18 User-defined types

PostgreSQL lets you add types that behave exactly like built-ins. There is a ladder of options, from merely constraining an existing type to writing a brand-new primitive (fig. 35). Examples extend `emp` from section 2.

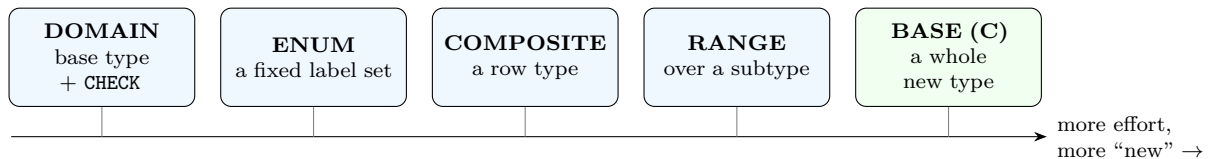


Figure 35: Five ways to extend the type system, from least to most work. A *domain* just constrains an existing type; *enum*, *composite*, and *range* build new structured types from existing ones; a *base type* (written in C with custom input/output functions) is a fully new primitive. Most needs are met by the first four.

18.1 Domains: a constrained existing type

A *domain* is a base type plus constraints, reusable as a named type — validation in one place:

```
1 CREATE DOMAIN positive_money AS numeric CHECK (VALUE >= 0);
2 CREATE DOMAIN email AS text CHECK (VALUE ~ '^[^@[:space:]]+@[^@[:space:]]+$');
3
4 ALTER TABLE emp ALTER COLUMN salary TYPE positive_money;    -- now negatives
   are rejected everywhere
```

18.2 Enums: a fixed set of labels

```
1 CREATE TYPE emp_status AS ENUM ('active', 'on_leave', 'terminated');
2 ALTER TABLE emp ADD COLUMN status emp_status NOT NULL DEFAULT 'active';
3
4 SELECT * FROM emp WHERE status = 'active';           -- type-checked; 'foo' would
   error
5 ALTER TYPE emp_status ADD VALUE 'retired' AFTER 'terminated'; -- labels are
   extensible
```

Enum values sort in *declaration order*, not alphabetically — often exactly what you want.

18.3 Composite types: a row type

A composite type bundles several fields into one value, usable as a column or a function return:

```
1 CREATE TYPE full_address AS (street text, city text, postcode text);
2
3 CREATE TABLE office (office_id int PRIMARY KEY, addr full_address);
4 INSERT INTO office VALUES (1, ROW('1 Main St', 'Springfield', '12345'));
5 SELECT (addr).city FROM office;           -- field access needs parentheses
```

(Every table also automatically has a composite type of the same name — its row type.)

18.4 Range types

Define a range over any ordered subtype:

```
1 CREATE TYPE salary_band AS RANGE (subtype = numeric);
2 SELECT salary_band(50000, 90000) @> 60000::numeric; -- true: 60k is in [50
   k,90k)
```

18.5 Base types (the C route)

A fully new primitive — its own storage and text representation — is a `CREATE TYPE` backed by C *input* and *output* functions (parse text → internal, and back), optionally with operators and index support. This is how extensions add types like `geometry` (PostGIS). It is advanced and rarely needed; the four structured options above cover most cases.

18.6 Making them first-class

Custom types can get their own operators (`CREATE OPERATOR`) and conversions (`CREATE CAST`), and they live in `pg_type` alongside the built-ins (section 17) — so functions, indexes, and casts treat them uniformly. **Takeaway.** Reach for a *domain* to enforce a rule once, an *enum* for a closed label set, a *composite* to group fields, and a *range* for intervals; drop to a C base type only when you genuinely need a new primitive.

19 Strings and text

Text handling in PostgreSQL is broad: concatenation, slicing, case/trim, search-and-replace, and full POSIX regular expressions, all over the `text` type (section 17). Examples use `emp.emp_name` from section 2.

19.1 Length and encoding

With a UTF-8 database, `char_length(s)` (alias `length`) counts *characters*, while `octet_length(s)` counts *bytes* — they differ for non-ASCII text.

```
1 SELECT length('café'), octet_length('café');      -- 4 characters, 5 bytes (é
    is 2 bytes)
```

19.2 Building and measuring

Function	Does
<code>a b, concat(a,b,...)</code>	concatenate (<code> </code> yields NULL if any side is NULL; <code>concat</code> skips NULLs)
<code>concat_ws(sep, ...)</code>	concatenate with a separator
<code>format('%s is %L', a, b)</code>	printf-style; %I quotes identifiers, %L quotes literals (injection-safe)
<code>repeat(s,n), reverse(s)</code>	repeat / reverse
<code>lpad(s,n,c), rpad(s,n,c)</code>	pad to width n

19.3 Case and trimming

```
1 SELECT upper('abc'), lower('ABC'), initcap('john doe');      -- ABC, abc, John
    Doe
2 SELECT btrim('  hi  '), ltrim('xxhi','x'), trim(both 'x' from 'xxhix'); -- '
    hi', 'hi', 'hi'
```

19.4 Slicing and finding (1-based)

Positions are 1-based (fig. 36).

```
1 SELECT substring('PostgreSQL' from 5 for 3);      -- 'gre'
2 SELECT left('PostgreSQL', 4), right('PostgreSQL', 3); -- 'Post', 'SQL'
3 SELECT position('S' in 'PostgreSQL'), strpos('PostgreSQL','S'); -- 8, 8
4 SELECT split_part('a,b,c', ',', 2);              -- 'b' (2nd field)
```

`substring('PostgreSQL' from 5 for 3) = 'gre'`

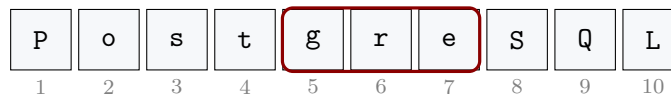


Figure 36: String positions are **1-based**. `substring(s from p for n)` starts at position `p` (the 5th character, `g`) and takes `n` characters. The same 1-based convention governs `left`, `right`, `position`, and array indexing — a frequent off-by-one trap for those used to 0-based languages.

19.5 Replace and translate

```

1 SELECT replace('a.b.c', '.', '/');           -- 'a/b/c' (literal substring)
2 SELECT translate('123', '123', 'abc');       -- 'abc' (per-character map)
3 SELECT regexp_replace('Alice42', '[0-9]+', '', 'g'); -- 'Alice' (regex, g =
    all)

```

19.6 Pattern matching

Operator / function	Meaning
LIKE / ILIKE	wildcard match (%=any run, _=one char); I = case-insensitive
~ ~* !~	POSIX regex match / case-insensitive / not-match
regexp_match(s, re)	first match's capture groups, as a text array
regexp_matches(s, re, 'g')	all matches, as a set of rows
regexp_split_to_array / _to_table	split on a regex into an array / rows

```

1 SELECT emp_name FROM emp WHERE emp_name ILIKE 'a%';           -- starts with a/A
    -> Alice
2 SELECT (regexp_match('salary: 90000', '(\d+)'))[1];         -- '90000'
3 SELECT regexp_split_to_array('a, b ,c', '\s*,\s*');          -- {a,b,c}

```

19.7 Splitting and aggregating

string_to_array / split_part split; string_agg (section 8) joins rows back into one string.

```

1 SELECT string_agg(emp_name, ', ' ORDER BY emp_id) FROM emp; -- 'Alice, Bob,
    Carol, Dave'

```

19.8 Collation and case-insensitivity

Ordering and comparison follow the column's *collation* (section 6); use COLLATE "C" for fast byte semantics. For case-insensitive matching, compare on lower(...) (and index it) or use the citext extension.

19.9 Indexing pattern searches (performance)

- Prefix LIKE 'abc%' can use a B-tree built with text_pattern_ops (or a COLLATE "C" column).
- Arbitrary substring / ILIKE / fuzzy needs a trigram index: CREATE EXTENSION pg_trgm; CREATE INDEX ON emp USING gin (emp_name gin_trgm_ops);

A leading-wildcard LIKE '%abc' cannot use a plain B-tree — that is what trigram indexes are for.

20 Glossary of symbols and terms

Terms

postmaster

the supervisor process that listens for connections and forks backends.

backend

the per-connection server process that runs a client's queries.

background processes

checkpointer, background writer, WAL writer, autovacuum, etc.

shared memory

the region (incl. `shared_buffers`, WAL buffers, locks) shared by all processes.

shared_buffers

PostgreSQL's in-memory cache of table/index pages.

WAL

write-ahead log: changes are logged before data files change; basis of recovery & replication.

planner / optimizer

the stage that turns a query tree into a cost-estimated plan tree.

executor

the stage that runs the chosen plan tree to produce rows.

GUC

"Grand Unified Configuration" — a PostgreSQL server/session parameter.

cross join

Cartesian product: every pairing of rows, no condition.

inner join

keeps only matching pairs; the default `JOIN`.

outer join

`LEFT`/`RIGHT`/`FULL`: keeps unmatched rows, `NULL`-padding the missing side.

semi join

left rows that have a match (via `EXISTS`); returns left columns once.

anti join

left rows with no match (via `NOT EXISTS`); beware the `NOT IN`/`NULL` pitfall.

self join

a table joined to itself via aliases (e.g. `employee/manager`).

lateral join

`JOIN LATERAL`: the right subquery may reference the left row; runs per outer row.

non-equi / theta join

a join whose condition is not equality (e.g. `BETWEEN`); cannot hash-join.

nested loop / hash / merge join

the three physical algorithms the executor uses for any logical join.

connection pooling

multiplexing many client connections onto few server connections.

PgBouncer

the de-facto external connection pooler for PostgreSQL.

pool_mode

when a server connection is returned: `session` / `transaction` / `statement`.

session pooling

server connection held for the whole client session; all features work, least sharing.

transaction pooling

server connection held per transaction; high density, breaks cross-transaction state.

statement pooling

server connection released after each statement; forbids multi-statement transactions.

psql

PostgreSQL's interactive terminal client (SQL plus backslash meta-commands).

meta-command

a backslash command run *inside* psql (`\d`, `\timing`, ...), not sent to the server.

\copy vs COPY

client-side (file on your machine) vs server-side (file on the server, needs privileges).

RETURNING

clause that makes INSERT/UPDATE/DELETE return affected rows.

upsert (ON CONFLICT)

insert, or update/skip on a unique-key conflict; EXCLUDED holds the proposed row.

COPY

bulk row stream, parsed once; the fast path for loading/unloading many rows.

ORDER BY

the only way to guarantee result order; rows are otherwise unordered.

NULLS FIRST/LAST

where NULLs sort; default LAST for ASC, FIRST for DESC.

collation

locale rules that determine text comparison/order; COLLATE "C" = byte order.

top-N / keyset pagination

ORDER BY ...LIMIT; keyset seeks past the last key instead of deep OFFSET.

Sort node / external merge

sorting step; spills to disk when it exceeds `work_mem`.

DISTINCT ON

PostgreSQL extension returning the first row per group per ORDER BY.

logical (clause) order

the fixed evaluation order FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT.

WHERE vs HAVING

row filter (before grouping, no aggregates) vs group filter (after grouping, aggregates allowed).

window function

computed in the SELECT phase (after grouping); cannot be used in WHERE/HAVING.

aggregate function

folds many rows into one value (count, sum, avg, ...); all but count(*) skip NULL.

FILTER

per-aggregate row filter: count(*) FILTER (WHERE ...).

ordered-set aggregate

... WITHIN GROUP (ORDER BY ...): percentiles, median, mode().

GROUPING SETS / ROLLUP / CUBE

multi-level grouping (subtotals, grand totals) in one query.

HashAggregate / GroupAggregate

hash-table vs sorted-stream grouping plans.

write-ahead rule

log a change (fsync) before its data page may be written to disk.

LSN

log sequence number: the monotonically increasing byte address of a WAL position.

checkpoint

flush all dirty buffers to data files; bounds WAL retained and recovery time.

REDO / crash recovery

replay WAL forward from the last checkpoint; idempotent via page LSN.

full-page write

first post-checkpoint change logs the whole page image (torn-page protection).

synchronous_commit

on fsyncs WAL before ack; off may lose recent commits but never corrupts.

wal_level

minimal / replica / logical: how much WAL is written.

PITR

point-in-time recovery: base backup plus archived WAL replayed to a target.

trigger

a function run automatically on table events (INSERT/UPDATE/DELETE/TRUNCATE).

BEFORE / AFTER / INSTEAD OF

trigger timing; INSTEAD OF is for views.

FOR EACH ROW / STATEMENT

trigger level: once per affected row vs once per statement.

NEW / OLD

the incoming / existing row inside a trigger function.

transition table

REFERENCING OLD/NEW TABLE: all affected rows as a relation (statement-level).

constraint trigger

an AFTER trigger that can be DEFERRABLE to commit time.

DELETE vs TRUNCATE

row-level removal (leaves dead tuples, fires triggers, WHERE) vs whole-table file swap (fast, frees space, ACCESS EXCLUSIVE).

dead tuple

a row marked deleted/old under MVCC; still occupies the file until VACUUM.

TRUNCATE ... RESTART IDENTITY / CASCADE

reset owned sequences / also truncate FK-referencing tables.

foreign key

a constraint that a child value must match a parent's PRIMARY KEY/UNIQUE (or be NULL).

referential action

ON DELETE/ON UPDATE: NO ACTION/RESTRICT/CASCADE/SET NULL/SET DEFAULT.

NOT VALID / VALIDATE CONSTRAINT

add a constraint cheaply, then validate existing rows later.

transaction

an atomic, isolated unit of work (BEGIN ... COMMIT/ROLLBACK).

ACID

atomicity, consistency, isolation, durability.

savepoint

a marker to ROLLBACK TO without aborting the whole transaction.

MVCC

multiversion concurrency control: updates write new row versions; readers see a snapshot.

snapshot

the set of row versions a transaction can see.

xmin / xmax

the transactions that created / retired a row version.

isolation level

READ COMMITTED / REPEATABLE READ / SERIALIZABLE: which anomalies are prevented.

serialization failure

SQLSTATE 40001 under SERIALIZABLE/REPEATABLE READ; retry the transaction.

FOR UPDATE / deadlock

explicit row lock; mutual lock waits are detected and one transaction aborts.

tuple

a physical row version, with header fields `xmin/xmax/ctid/hint` bits.

HOT update

heap-only tuple: an update with no indexed-column change on the same page, skipping new index entries.

VACUUM / VACUUM FULL

reclaim dead-tuple space for reuse (online) / rewrite the table compactly (exclusive lock).

visibility map

per-table map of all-visible pages; enables index-only scans and vacuum skipping.

freezing / wraparound

stamping old tuples permanently visible before 32-bit xids wrap; advances `relfrozenxid`.

window function

computes per-row over a frame (`OVER (PARTITION BY ... ORDER BY ...)`); keeps all rows.

frame (ROWS / RANGE)

the subset of a partition feeding a window function; default is `RANGE ... CURRENT ROW`.

partitioning

splitting a table into RANGE/LIST/HASH partitions; enables pruning and slice management.

partition pruning

the planner scanning only partitions a query's key predicate can match.

numeric vs float

`numeric` is exact (use for money); `real/double` are inexact binary floats.

timestampz

absolute instant stored in UTC; preferred over zone-less `timestamp`.

jsonb

binary, indexable JSON (`@>`, `->`, GIN); prefer over `json`.

cast

`x::t` / `CAST(x AS t)`; implicit / assignment / explicit strengths.

domain

a base type plus `CHECK` constraints, reused as a named type.

enum / composite / range type

user-defined: label set / row type / interval over a subtype.

LIKE / regex (~)

wildcard vs POSIX pattern matching; trigram (`pg_trgm`) GIN indexes substring/`ILIKE`.

1-based strings

substring/position/array indexing start at 1.

Symbols

$\pi, \sigma, \bowtie$
relational-algebra projection, selection, natural join.

R, S
relations (tables, views, or subquery results).

$|R|$
cardinality: the number of rows in R .

s selectivity of a predicate, $s \in [0, 1]$.